VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING

Development of a Vision-Based Human-Following Robot with Fall Detection and Real-Time Alert System

FINAL-TERM ESSAY
AUTOMATION AND CONTROL ENGINEERING

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING



Development of a Vision-Based Human-Following Robot with Fall Detection and Real-Time Alert System

FINAL-TERM ESSAY
AUTOMATION AND CONTROL ENGINEERING

Advisor
Dr. Tran Thanh Nam

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

First of all, I would like to express my sincere gratitude to Dr. Tran Thanh Nam for his dedicated guidance, valuable advice, and continuous support throughout the implementation of this embedded project. His enthusiasm, experience, and encouragement have greatly contributed to the successful completion of this project.

I would also like to thank the Faculty of Electrical and Electronics Engineering, Ton Duc Thang University for providing favorable learning conditions, facilities, and academic support during the project development process.

In addition, I am deeply grateful to my friends, who were always willing to help, share knowledge, and support me whenever I encountered difficulties. Their encouragement and cooperation helped me overcome many challenges and maintain a positive attitude during the implementation of this project.

Finally, I would like to express my heartfelt thanks to my family for their constant love, encouragement, and support. Their care and motivation have been an important source of strength, helping me complete this project successfully.

Ho Chi Minh city, May 17th, 2026

Author

DECLARATION OF AUTHORSHIP

I hereby declare that this essay was completed by myself under the guidance and supervision of Dr. Tran Thanh Nam. The content, results, data, and figures presented in this essay are original, have been independently prepared by me, and have not been submitted elsewhere for any previous academic purpose.

All materials, comments, evaluations, and references taken from other authors or organizations have been properly acknowledged and cited in the reference section.

I take full responsibility for the academic integrity of this essay. Ton Duc Thang University shall not be held responsible for any copyright infringement or academic violation caused by my work, if any.

Ho Chi Minh City, May 17th, 2026

Author

TABLE OF CONTENTS

TABLE OF FIGURES ..... **VI**

LIST OF ACRONYMS ..... **VIII**

CHAPTER 1. INTRODUCTION **1**

1.1 Background of the Project 1

1.2 Motivation of the Project 1

1.3 Project Objectives 2

1.4 Social Impact and Applications 3

CHAPTER 2. THEORETICAL BACKGROUND **5**

2.1 Overview of IoT Architecture 5

2.2 Edge AI and Computer Vision in IoT Systems 6

2.3 Human Detection and Human State Classification Using YOLO 9

2.4 Embedded Control Systems in IoT Robots 12

2.5 IoT Communication Protocols 14

2.6 Alert Management, Dashboard Monitoring, and Safety Response 17

2.7 Evaluation Metrics: FPS, Latency, Detection Accuracy, Alert Response Time, and System Reliability 19

CHAPTER 3. SYSTEM IMPLEMENTATION **23**

3.1 Overview of the Proposed System Architecture 23

3.2 Experimental Environment and Testbed Setup 24

3.3 Hardware Design of the Robotic Vehicle 27

3.4 Edge AI Software Design on Raspberry Pi 29

3.5 Backend Server and Database Design 32

3.6 IoT Data Pipeline and Control Flow 35

3.7 Web Dashboard Design and User Interface 37

3.8 Security Implementation 41

3.9 Safety Response and Alert Generation Logic 44

3.10 Algorithm-to-Hardware Mapping 47

CHAPTER 4. SYSTEM RESULTS AND EVALUATION 50

4.1 Experimental Setup 50

4.2 Testing Scenarios 51

4.3 Results of the Edge AI Processing Pipeline 52

4.4 Fall Detection and Alert Generation Results 53

4.5 System Performance Evaluation 53

4.6 Discussion, Limitations, and Improvement Directions 54

CHAPTER 5. CONCLUSION AND FUTURE WORK 55

5.1 Conclusion 55

5.2 Practical Applicability of the System 55

5.3 Future Development Directions 56

REFERENCES 58

TABLE OF FIGURES

Figure 2.1. Overall architecture of the AI-IoT human-following robot with fall detection and real-time alert system.....6
Figure 2.2. Layered decomposition of the proposed AI-IoT system. 6
Figure 2.3. Edge AI Pipeline diagram.....8
Figure 2.4. YOLO-based detection and classification pipeline for movement control and alert generation 10
Figure 2.5. Embedded control flow in the proposed AI-IoT robotic car system 13
Figure 2.6. Communication links between Raspberry Pi, Arduino, backend server, and web dashboard.....15
Figure 2.7. Alert management and safety response flow in the proposed system 18
Figure 2.8. Latency Breakdown Diagram 21
Figure 3.1. Overall implementation architecture of the proposed AI-IoT robotic car system.....24
Figure 3.2. Experimental testbed setup of the proposed AI-IoT robotic car system. 25
Figure 3.4. Edge AI software pipeline on Raspberry Pi for person detection and human state classification 31
Figure 3.5. Backend server and database architecture of the proposed system.....33
Figure 3.6. End-to-end IoT data pipeline and control flow of the proposed system 37
Figure 3.7. Web dashboard interface for monitoring device status, events, and alerts 38
Figure 3.8. Authentication and authorization flow in the proposed system 42
Figure 3.9. Safety response and alert generation flow of the proposed system.....45
Figure 4.1. Log in to the dashboard 50
Figure 4.2. Experimental setup for testing the proposed AI-IoT robotic car system51
Figure 4.3. Main testing scenarios of the proposed system when person turns left .51
Figure 4.4. Main testing scenarios of the proposed system when person falls.....52

Figure 4.5. Application interface 52
Figure 4.6. Fall detection and alert generation workflow 53
Table 4.7. System performance evaluation metrics 53

LIST OF ACRONYMS

AI	Artificial Intelligence
API	Application Programming Interface
ADMIN	Administrator
bcrypt	Blowfish Crypt (password hashing algorithm)
CSI	Camera Serial Interface
DC	Direct Current
FPS	Frames Per Second
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
IP	Internet Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
MJPEG	Motion Joint Photographic Experts Group
MQTT	Message Queuing Telemetry Transport
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PATCH	Partial Update (HTTP method)
POST	Submit Data (HTTP method)
PWM	Pulse Width Modulation
REST	Representational State Transfer
RFC	Request for Comments
RPi	Raspberry Pi

SQL	Structured Query Language
TCP	Transmission Control Protocol
UART	Universal Asynchronous Receiver - Transmitter
URL	Uniform Resource Locator
USB	Universal Serial Bus
USER	Standard user role
Wi-Fi	Wireless Fidelity
YOLO	You Only Look Once

CHAPTER 1. INTRODUCTION

1.1 Background of the Project

In recent years, the Internet of Things (IoT) has become an important foundation for developing smart systems that connect physical devices, collect data, and respond to real-world situations through a network. When IoT is combined with artificial intelligence, especially computer vision, the system can not only sense the environment but also analyze images, recognize human activities, and react in near real time.

Fall detection is an important problem in healthcare and human-support applications, especially for elderly people, patients, and people living alone. According to the World Health Organization (WHO), falls are the second leading cause of unintentional injury deaths worldwide, and adults older than 60 years of age suffer the greatest number of fatal falls. Therefore, a delayed response after a fall may lead to serious health consequences. Traditional fixed cameras are limited by viewing angles and monitoring range, while wearable sensors depend on whether the user wears them properly.

Based on this background, the project “Development of a Vision-Based Human-Following Robot with Fall Detection and Real-Time Alert System” is developed as an AI-IoT robotic car model. The robot uses a camera to detect and follow a person, identifies human states such as standing, sitting, and falling, and sends alerts to a web-based monitoring system. The prototype is built using a Raspberry Pi 5, Pi Camera V3, Arduino Uno R3, and L298N motor driver, showing how IoT architecture and protocols can be applied in a practical end-to-end system from sensing and edge processing to communication, server handling, and user monitoring.

1.2 Motivation of the Project

The idea for this project comes from a real and practical need. In many households and healthcare facilities, elderly people or patients often move around without

constant supervision. When a fall occurs, the person may not be able to call for help immediately, and if no one is nearby to notice, the consequences can become much more serious. This situation raises a clear question: how can a system automatically detect a fall and send an alert without requiring someone to watch a screen at all times?

At the same time, advances in affordable single-board computers, embedded controllers, and open-source AI models have made it possible to build intelligent systems without expensive specialized hardware. A Raspberry Pi 5 is powerful enough to process camera data and run YOLO-based detection models, while an Arduino Uno R3 can handle vehicle control reliably at low cost. Together with the Pi Camera V3 and L298N motor driver, this combination makes it feasible for a student project to cover a complete IoT pipeline — from physical sensing and edge AI processing to server communication and a web-based dashboard.

Another motivation is the limitation of existing approaches. Fixed cameras cannot follow a moving person, and wearable devices require the user to remember to wear them properly. A mobile robot that actively follows a person and continuously monitors their condition can address both of these limitations in a single platform.

Finally, this project serves as a practical application of the IoT Architecture and Protocols course. By building a system that spans the perception layer, edge processing layer, communication layer, service layer, data layer, application layer, and security layer, the project demonstrates how theoretical IoT concepts can be translated into a working end-to-end system.

1.3 Project Objectives

The main objective of this project is to develop a vision-based AI-IoT robotic car that can detect and follow a person, recognize human states, and send alerts when an abnormal situation occurs. The system is designed as an end-to-end IoT model, covering physical sensing, edge AI processing, vehicle control, server communication, data storage, and web-based monitoring.

In this project, the robotic car uses a Pi Camera V3 connected to a Raspberry Pi 5 to capture visual data from the environment. The Raspberry Pi works as the edge processing device, where YOLO-based models are used to detect a person and classify human states such as standing, sitting, and fallen. Based on the detected position and state, the Raspberry Pi generates suitable movement commands for the vehicle.

For vehicle control, an Arduino Uno R3 receives commands from the Raspberry Pi through serial communication. The L298N motor driver is used to control the DC motors, allowing the car to move forward, turn left, turn right, search for the person, or stop when needed. This separation between AI processing and motor control makes the system easier to manage and more reliable.

Another objective is to build a backend and dashboard system for monitoring and alert management. The Raspberry Pi sends IoT events to the NestJS backend through HTTP REST API in JSON format. The server stores device events, updates device status, and creates alerts for critical events such as FALL_DETECTED or PERSON_LOST. The Next.js dashboard then displays device status, recent events, and alerts for users.

To protect the system, basic security measures are also implemented, including JWT-based authentication, bcrypt password hashing, and role-based access control for ADMIN and USER roles. Overall, this project demonstrates how IoT architecture and protocols can be applied in a practical system by combining perception, edge processing, communication, service, data, application, and security layers into one working AI-IoT solution for human-following and fall detection.

1.4 Social Impact and Applications

The system developed in this project has practical value beyond the academic scope. By combining a mobile robot with AI-based fall detection and real-time alert capabilities, the project addresses real needs in human safety monitoring.

In home care and elderly support, the robot can follow an elderly person or a patient around the living space and notify a caregiver through the web dashboard when a fall is detected. This can help reduce the risk of delayed response when no one is physically present to provide immediate assistance.

In healthcare facilities such as hospitals or rehabilitation centers, the system can support medical staff by monitoring patients with limited mobility or a high risk of falling. Instead of requiring continuous manual observation, the system can automatically report abnormal events and help staff respond more quickly.

A similar approach can also be extended to industrial and workplace environments, where workers may operate in hazardous areas. If a worker falls or becomes unresponsive, the system can trigger an alert to a safety officer without requiring manual reporting.

Beyond these use cases, the project demonstrates the broader potential of combining edge AI, IoT communication, backend services, and web-based monitoring in one practical system. Future extensions may include multi-person tracking, mobile push notifications, voice interaction, or integration with emergency response services. Overall, the project shows that IoT and AI can be applied in an affordable and practical way to improve human safety, reduce response time in emergency situations, and support caregivers in both home and professional environments.

CHAPTER 2. THEORETICAL BACKGROUND

2.1 Overview of IoT Architecture

The Internet of Things (IoT) refers to a network of physical devices embedded with sensors, processors, and communication modules. These devices can collect data from the environment, exchange information through a network, and perform actions based on processed results. An IoT system is usually organized as a connected architecture in which physical devices, edge processing units, communication protocols, backend services, databases, and user applications work together to support real-time monitoring and control.

In this project, the system follows an IoT architecture that covers the main stages from physical sensing to user-facing applications. The Pi Camera V3 captures image and video data from the environment and sends them to the Raspberry Pi 5, which acts as the edge processing device. The Raspberry Pi runs YOLO-based AI models to detect a person and classify human states, then sends movement commands to an Arduino Uno R3 through serial communication. The Arduino controls the DC motor through the L298N motor driver and controls the steering servo separately using PWM signals.

When an important event is detected, such as a fall or a person being lost from the camera view, the Raspberry Pi sends structured event data to a NestJS backend server through an HTTP REST API over the local network. The server stores this data in a SQLite database managed by Prisma ORM, applies business logic to generate alerts, and exposes REST API endpoints for the web dashboard. The Next.js dashboard allows authorized users to monitor device status, view event history, and manage alerts with role-based access control.

This architecture forms the basis for the following sections, which discuss edge AI and computer vision, YOLO-based human detection and human state classification, embedded control, IoT communication protocols, alert management, safety response, and evaluation metrics in this project.

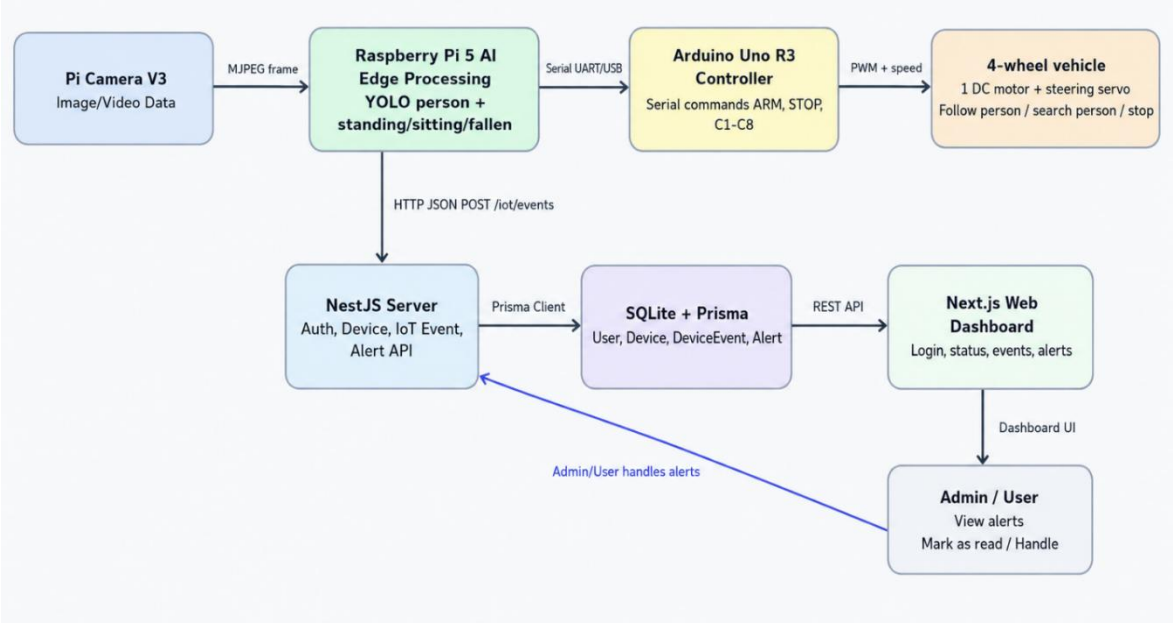


Figure 2.1. Overall architecture of the AI-IoT human-following robot with fall detection and real-time alert system.

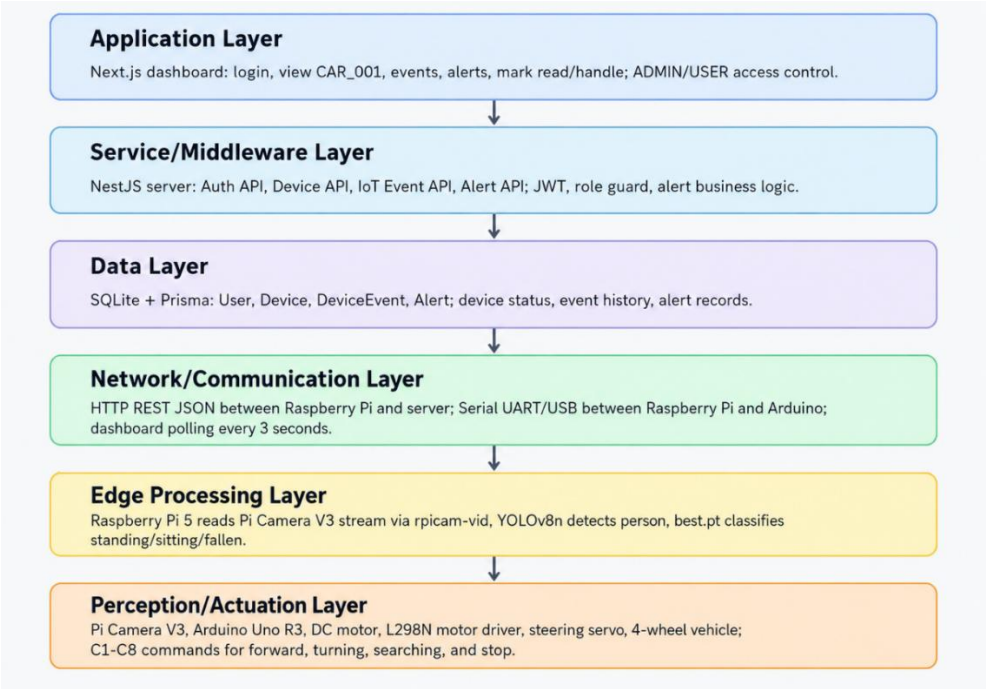


Figure 2.2. Layered decomposition of the proposed AI-IoT system.

2.2 Edge AI and Computer Vision in IoT Systems

Edge AI refers to the deployment of artificial intelligence models directly on local devices, rather than sending raw data to a remote server for processing. In IoT

systems, this approach is important because it reduces network bandwidth usage, minimizes response latency, and allows the system to continue basic operation even when the network connection is unstable. Instead of transmitting large amounts of raw sensor data to the cloud, the edge device processes the data locally and only sends final results or structured event data to the server.

Computer vision is one of the most common applications of Edge AI in IoT systems. It allows devices to capture images or video from a camera, analyze visual content in near real time, and extract meaningful information such as the presence of a person, their location in the frame, or their physical state. This capability is especially useful in monitoring and safety applications, where fast and accurate visual analysis is required.

In this project, Edge AI and computer vision are implemented on the Raspberry Pi 5, which serves as the edge processing device. The Pi Camera V3 continuously captures video frames and passes them to the Raspberry Pi through the MJPEG stream generated by rpicam-vid. Two YOLO-based models are then applied to process the frames. The first model, yolov8n.pt, is used to detect the presence of a person and determine their position within the camera frame. The second model, best.pt, is a custom-trained model that identifies the human state as standing, sitting, or fallen. By running both models locally on the Raspberry Pi, the system can make movement and alert decisions without depending on a remote server for AI inference.

This Edge AI approach brings several practical benefits to the project. It reduces the amount of data transmitted over the network because only structured event data, such as event type, human state, confidence score, and vehicle command, are sent to the backend instead of raw video. It also lowers response latency, allowing the robot to react faster when a fall is detected or when the target person is lost from the camera view.

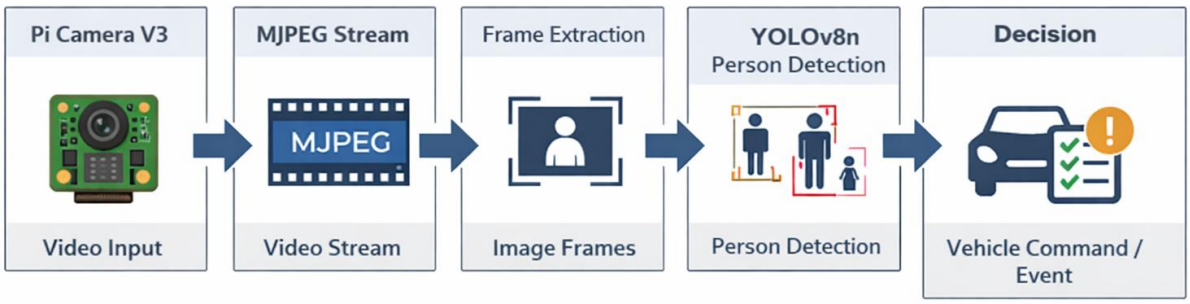


Figure 2.3. Edge AI Pipeline diagram

Furthermore, local processing allows the robot to continue its basic detection and control tasks even if the connection to the backend server is temporarily unavailable.

Table 2.1. Edge AI and computer vision processing pipeline in the proposed system

Function	Role in the System	Component Used
Edge AI processing	Runs AI inference locally on Raspberry Pi 5	Raspberry Pi 5
Computer vision	Extracts human-related information from camera frames	Pi Camera V3 + OpenCV
Person detection	Detects the presence and position of a person	yolov8n.pt
Human state classification	Classifies the human state as standing, sitting, or fallen	best.pt

Event reporting	Sends structured event data to the backend instead of raw video	HTTP REST API
Safety response	Supports fast stopping and alert generation when a fall is detected	Arduino + NestJS

2.3 Human Detection and Human State Classification Using YOLO

YOLO, which stands for You Only Look Once, is a family of real-time object detection models that can process an image in a single forward pass through a neural network. Unlike traditional detection approaches that analyze multiple regions of an image separately, YOLO predicts bounding boxes, class labels, and confidence scores for detected objects in one stage. This makes YOLO suitable for real-time applications, especially when the system runs on resource-constrained edge devices.

YOLOv8, developed by Ultralytics, is a modern version in the YOLO family. It provides good detection performance, fast inference speed, and convenient deployment. YOLOv8 supports different computer vision tasks, including object detection, image classification, and pose estimation. The nano variant, YOLOv8n, is designed for lightweight deployment, offering a balance between speed and accuracy. This makes it suitable for real-time inference on embedded platforms such as the Raspberry Pi.

In this project, two YOLO-based models are used together to perform human detection and human state classification. The first model, yolov8n.pt, is a pre-trained YOLOv8 nano model used to detect the presence of a person and determine their horizontal position in the camera frame. Based on the position of the detected bounding box relative to the left, center, and right zones of the frame, the Raspberry Pi generates an appropriate movement command for the vehicle. The second model,

best.pt, is a custom-trained classification model with three output classes: standing, sitting, and fallen. This model is used to determine the current physical state of the detected person. When the fallen class is predicted with sufficient confidence, the system sends a stop command to the Arduino and reports a FALL_DETECTED event to the backend server.

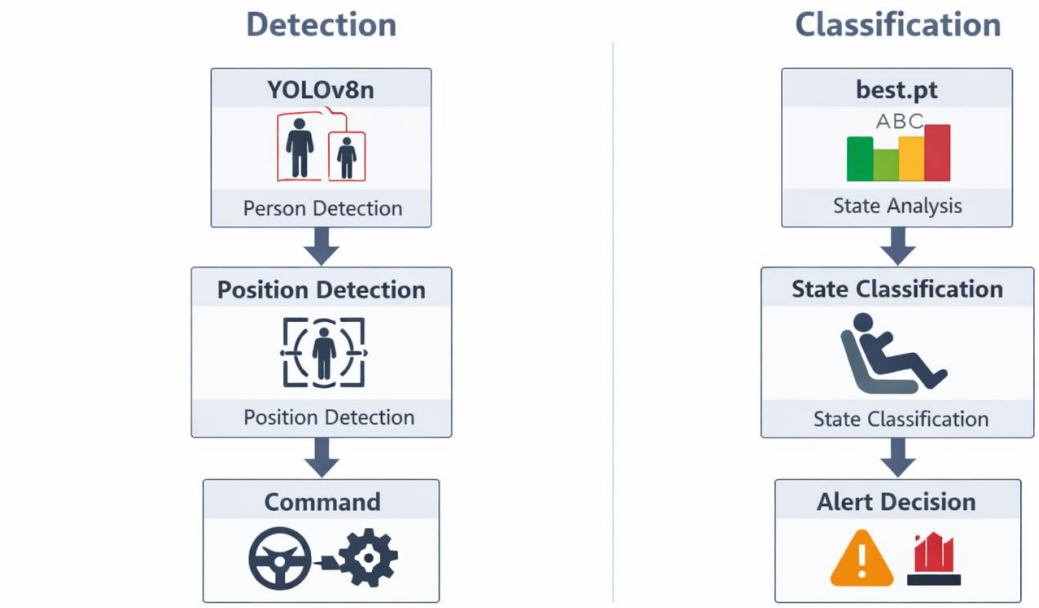


Figure 2.4. YOLO-based detection and classification pipeline for movement control and alert generation

To maintain near real-time performance on the Raspberry Pi, several optimization techniques have been applied. A dedicated thread is used to continuously read the latest frame from the MJPEG stream and discard older frames to reduce processing delays. The input image size for YOLO inference is reduced to lower the computational load. The state classification model is applied every N frames instead of every frame, because human state changes more slowly than position. A smoothing mechanism is also applied to the detected center position to reduce command instability caused by small movements or detection noise.

The YOLO-based processing flow in this project can be summarized as follows: Pi Camera V3 → MJPEG stream via rpicam-vid → frame extraction → YOLOv8n

person detection → best.pt human state classification → decision logic → vehicle command or IoT event.

Table 2.2. YOLO-based detection and classification tasks in the proposed system.

Task	Model / Method	Output	Role in the System
Person detection	yolov8n.pt	Person bounding box and confidence score	Locates the target person in the camera frame.
Position estimation	Bounding box center calculation	Left / center / right region	Generates vehicle movement direction.
Human state classification	best.pt	Standing / sitting / fallen	Identifies the physical state of the person.
Fall detection	Fallen class prediction	FALL_DETECTED event	Stops the robot and triggers an alert.
Person-lost handling	No person detected	PERSON_LOST event	Starts searching for behavior or reports person lost.

2.4 Embedded Control Systems in IoT Robots

An embedded control system is a dedicated computing unit designed to perform specific control functions within a larger system. In robotics and IoT applications, embedded controllers are responsible for receiving commands from a higher-level processing unit, interpreting those commands, and directly driving actuators such as motors, servos, and relays. Unlike general-purpose computers, embedded controllers are optimized for real-time responsiveness, low power consumption, and reliable operation in hardware-based environments.

In mobile robot systems, a common design pattern is to separate the AI processing layer from the motor control layer. The high-level processor handles perception and decision-making, while a dedicated embedded controller handles low-level actuation. This separation makes the system more modular, easier to debug, and more reliable because each part has a clear responsibility.

Serial communication is one of the most widely used methods for connecting a high-level processor to an embedded controller in IoT robotics. The UART protocol transmits data as a sequence of bits at a fixed baud rate. It is simple, reliable, and supported by most microcontrollers and single-board computers. In practice, UART communication can also be implemented through a USB serial connection, which provides a convenient physical link between devices.

PWM, or Pulse Width Modulation, is a common method for controlling servo motors and motor speed in embedded systems. A PWM signal is a digital square wave with a fixed frequency but variable duty cycle. For servo motors, the pulse width determines the rotation angle. For DC motors controlled by a motor driver such as the L298N, the PWM duty cycle can be used to adjust the motor speed.

In this project, the embedded control system is built around an Arduino Uno R3, which receives movement commands from the Raspberry Pi 5 through a USB serial connection using the UART protocol at a baud rate of 9600. The command set includes ARM to enable vehicle operation, STOP to disable it, and directional commands from C1 to C8 to specify movement behavior. After receiving a command,

the Arduino generates PWM signals to control the steering servo angle and sends direction and speed control signals to the L298N motor driver, which drives the DC motor accordingly. This design allows the Raspberry Pi to focus on AI inference and event reporting, while the Arduino handles time-sensitive motor and steering control tasks.

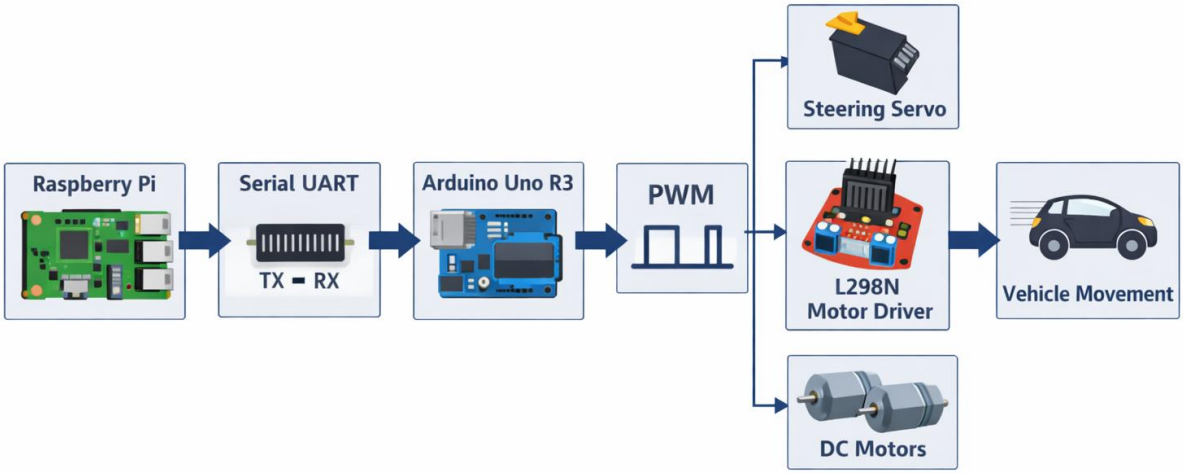


Figure 2.5. Embedded control flow in the proposed AI-IoT robotic car system

Table 2.3. Serial command set for embedded vehicle control

Command	Description	Use Case
ARM	Enable vehicle operation	Sent at system startup
STOP	Disable vehicle operation	Sent at system shutdown
C1, speed	Move forward	Person detected at the center
C3, speed	Turn left	Person slightly on the left
C4, speed	Turn right	Person slightly on the right
C6, speed	Sharp left turn	Person far left or searching

C7, speed	Sharp right turn	Person far right or searching
C8	Stop immediately	Fall detected, sitting, timeout, or unsafe state

2.5 IoT Communication Protocols

Communication protocols define the rules and standards that allow different devices and systems to exchange data reliably. In IoT systems, multiple communication protocols are usually used at the same time, with each protocol serving a different purpose depending on distance, data type, speed requirement, and reliability. Choosing the right protocol for each communication link is an important design decision because it directly affects the overall performance and stability of the system.

In this project, the main communication protocols include Serial UART, Wi-Fi, TCP/IP, and HTTP REST. Serial UART is used for communication between the Raspberry Pi and the Arduino. Wi-Fi and TCP/IP are used for network communication between the Raspberry Pi, backend server, and web dashboard. HTTP REST is used for structured data exchange between the edge device, backend, and dashboard.

Serial UART is used for the communication link between the Raspberry Pi 5 and the Arduino Uno R3. UART, which stands for Universal Asynchronous Receiver-Transmitter, transmits data as a sequence of bits through TX/RX communication lines at a fixed baud rate, which is set to 9600 in this project. The Raspberry Pi sends short text-based commands such as ARM, STOP, C1,10, and C8 to the Arduino through a USB serial connection. UART is chosen for this link because it is simple, low-latency, and supported by both the Raspberry Pi and Arduino without requiring additional network infrastructure.

Wi-Fi provides the wireless network connection between the Raspberry Pi and the backend server. The Raspberry Pi connects to the local Wi-Fi network, allowing

it to communicate with the NestJS backend server over the local IP network. Wi-Fi is suitable for this link because it provides enough bandwidth for sending structured JSON event data and does not require a physical cable between the moving robot and the server.

TCP/IP is the underlying network and transport protocol stack used for network communication in this project. IP provides addressing and routing, allowing the Raspberry Pi, backend server, and dashboard client to identify and reach each other within the local network. TCP provides reliable, ordered, and error-checked data delivery between applications. Since the system sends important event data such as FALL_DETECTED and PERSON_LOST, reliable delivery is necessary to reduce the risk of missing critical alerts.

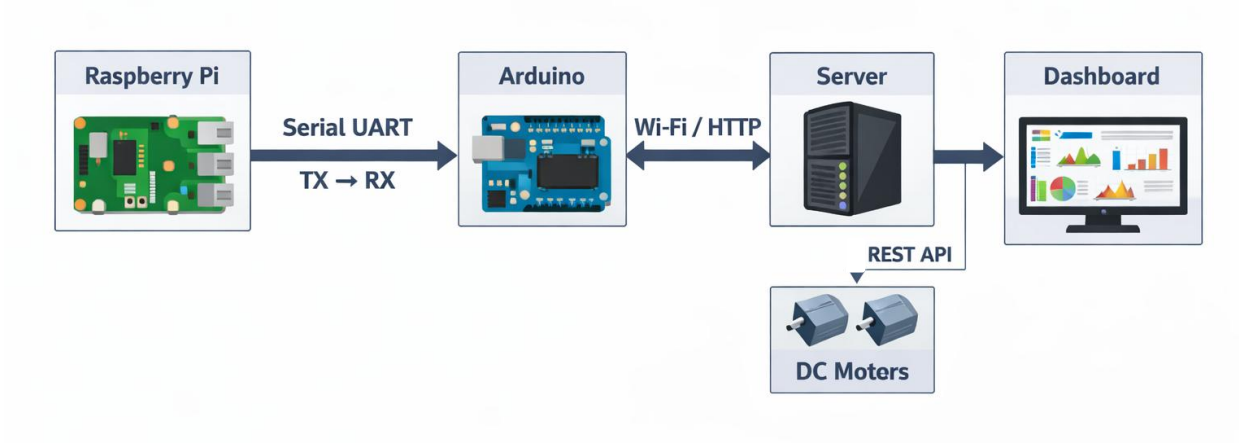


Figure 2.6. Communication links between Raspberry Pi, Arduino, backend server, and web dashboard

HTTP REST is the application-level communication method used between the Raspberry Pi and the NestJS backend, as well as between the Next.js dashboard and the backend. The Raspberry Pi sends IoT event data to the server by making HTTP POST requests to the /iot/events endpoint. The JSON payload contains information such as deviceCode, eventType, humanState, confidence, vehicleCommand, and message. The dashboard retrieves device status, event history, and alert data by sending HTTP GET requests to the corresponding API endpoints. HTTP REST is chosen because it is stateless, widely supported, easy to implement, and compatible with the NestJS backend framework.

Overall, these protocols allow the system to connect the embedded control layer, edge AI layer, backend server, and user dashboard into one complete IoT workflow. Serial UART supports low-level vehicle control, Wi-Fi and TCP/IP provide network connectivity, and HTTP REST enables structured event reporting and monitoring through the backend and dashboard.

Table 2.4. Transport-related communication in the proposed system

Protocol / Technology	Communication Path	Role in the System
Serial UART/USB	Raspberry Pi 5 → Arduino Uno R3	Sends control commands such as ARM, STOP, and C1–C8.
Wi-Fi	Raspberry Pi 5 → Local network	Connects the robot to the backend server wirelessly.
TCP/IP	Raspberry Pi / Server / Dashboard	Provides addressing, routing, and reliable network communication.
HTTP REST	Raspberry Pi 5 → NestJS backend	Sends IoT event data to /iot/events.
JSON	Raspberry Pi 5 → NestJS backend	Structures event data such as device code, human state, confidence, and command.
REST API polling	Next.js dashboard → NestJS backend	Updates device status, recent events, and alerts.

2.6 Alert Management, Dashboard Monitoring, and Safety Response

In IoT systems designed for safety monitoring, the ability to detect abnormal events, generate alerts, and present them to users in a timely and organized manner is a critical requirement. An alert management system defines how events are captured, classified, stored, and communicated to the appropriate users. A well-designed alert lifecycle ensures that critical events can be tracked from the moment they are generated until they are resolved.

A dashboard monitoring system provides a user-facing interface that aggregates data from multiple sources and presents it in a readable and actionable format. In IoT applications, dashboards commonly display device status, recent event history, and active alerts. They may also allow users to perform actions such as acknowledging, reading, or resolving an alert. Role-based access control is often applied to ensure that each user can only view and manage the data they are authorized to access.

Safety response refers to the automatic actions that a system performs when a critical event is detected. In a human monitoring system, safety response may include stopping a vehicle, triggering an alert, notifying a caregiver, or logging the event for later review. The speed and reliability of this response are important because delayed handling may reduce the effectiveness of the safety monitoring system.

In this project, alert management and safety response are implemented across three main components: the Raspberry Pi, the NestJS backend, and the Next.js dashboard. When the Raspberry Pi detects a critical event such as FALL_DETECTED or PERSON_LOST, it sends a structured JSON event to the backend through an HTTP POST request. The backend receives this event, stores it as a DeviceEvent record in the SQLite database, updates the device status, and creates an Alert record if the event type meets the alert generation criteria.

Each alert follows a simple lifecycle with three states: NEW, READ, and HANDLED. A NEW alert is created when a critical event is first detected. After a user views the alert, it can be marked as READ. When the issue has been checked or

resolved, the alert can be marked as HANDLED. This lifecycle helps organize warning information and prevents important alerts from being ignored.

The Next.js dashboard provides the user interface for monitoring and managing these alerts. Users can log in, view the current status of the robotic vehicle, browse recent events, and check active alerts. The dashboard refreshes its data every three seconds by polling the backend API, which provides near real-time visibility into the system state. Users can mark alerts as read or handled directly from the dashboard. Access to device and alert data is controlled by role-based permissions, where ADMIN users can manage all devices and alerts, while USER accounts can only access assigned data.

On the vehicle side, safety response is handled locally before the event is reported to the server. When a fallen state is detected, the Raspberry Pi sends a C8 stop command to the Arduino through the serial connection. When the target person is lost from the camera view, the robot may enter a searching behavior first and stop after a timeout if the person is still not found. This design ensures that the robot can respond to unsafe situations locally, while the backend and dashboard provide event recording, alert management, and user monitoring.



Figure 2.7. Alert management and safety response flow in the proposed system

Table 2.5. Alert events and safety responses in the proposed system

Event / State	Vehicle Response	Backend / Dashboard Response
PERSON_STANDING	Follow person based on position	Update event/status

PERSON_SITTING	Stop for safety	Record event
FALL_DETECTED	Send C8 stop command	Create critical alert
PERSON_LOST	Search first, then stop after timeout	Create alert/event
NEW	—	Alert has been created
READ	—	User has viewed the alert
HANDLED	—	User has resolved/processed the alert

2.7 Evaluation Metrics: FPS, Latency, Detection Accuracy, Alert Response Time, and System Reliability

Evaluation metrics are measurable indicators used to assess whether a system meets its design objectives and performs correctly under real operating conditions. In IoT systems that combine AI processing, embedded control, network communication, and user-facing applications, multiple metrics are needed to evaluate different aspects of the system independently. Each metric focuses on a specific component or interaction point in the overall data flow.

Frames Per Second (FPS) measures the rate at which the AI processing pipeline can analyze video frames from the camera. In this project, FPS reflects the performance of the Raspberry Pi 5 when running yolov8n.pt for person detection and best.pt for human state classification. A higher FPS allows the system to detect changes in human position and state more frequently, which can improve vehicle following and fall detection responsiveness. Therefore, FPS is an important indicator of whether the edge AI system can operate in near real time.

Latency refers to the time delay between an event occurring in the physical world and the system responding to it. In this project, latency can be considered at two

levels. The first is control latency, which is the delay from detecting a change in the person's position to sending and executing the corresponding vehicle command through the Serial UART connection. The second is alert latency, which is the delay from detecting a critical event to sending the event data to the backend server. Lower latency helps the robot follow the person more smoothly and respond more quickly to unsafe situations.

Detection accuracy measures how correctly the YOLO-based models identify persons and classify human states. For person detection using yolov8n.pt, accuracy refers to how reliably the model detects a person under different lighting conditions, distances, and camera angles. For human state classification using best.pt, accuracy refers to how correctly the model distinguishes between standing, sitting, and fallen states. In fall detection, false negatives are especially critical because a real fall that is not detected may lead to a missed alert and delayed safety response.

Alert response time measures the total time from when a critical event such as FALL_DETECTED or PERSON_LOST is detected by the Raspberry Pi to when the alert is created in the backend database and becomes visible on the dashboard. This metric covers the full pipeline, including AI inference, HTTP POST transmission, server-side processing, database writing, and dashboard polling. A shorter alert response time means caregivers or monitoring users can be notified more quickly when an emergency occurs.

System reliability refers to the ability of the system to operate correctly and consistently over an extended period without failure. In this project, reliability can be evaluated by observing whether the vehicle follows the person without frequently losing track, whether alerts are consistently generated for fall or person-lost events, whether the backend correctly stores events and updates device status, and whether the dashboard accurately reflects the current system state. Reliability also includes the stability of the Serial UART connection between the Raspberry Pi and Arduino, as well as the Wi-Fi connection between the Raspberry Pi and the backend server.

The project file also describes this complete workflow from AI processing and Arduino control to backend storage and dashboard monitoring.

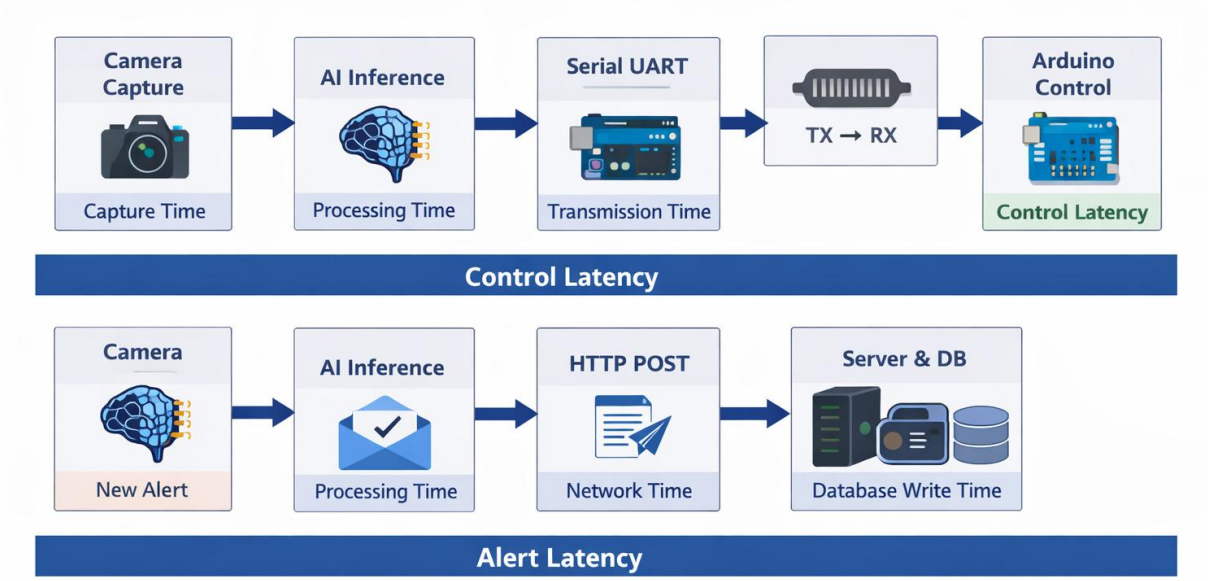


Figure 2.8. Latency Breakdown Diagram

The main evaluation metrics used in this project are summarized in Table 2.6.

Table 2.6. Evaluation metrics for the proposed AI-IoT robotic car system

Metric	Description	Importance in the Project
FPS	Number of video frames processed per second.	Indicates whether the Raspberry Pi can perform near real-time AI processing.
Control latency	Delay from AI decision to vehicle command execution.	Affects human-following smoothness and stopping response.

Alert response time	Time from critical event detection to dashboard alert display.	Measures how quickly users can be notified.
Detection accuracy	Correctness of person detection and state classification.	Reduces missed detections and false alerts.
System reliability	Stability of camera, AI model, Serial UART, Wi-Fi, backend, and dashboard.	Ensures the system can operate consistently during monitoring.

CHAPTER 3. SYSTEM IMPLEMENTATION

3.1 Overview of the Proposed System Architecture

The proposed system is an AI-IoT robotic car designed to detect and follow a person, recognize human states, and send near real-time alerts to a web-based monitoring dashboard. The system is structured as a complete end-to-end IoT solution that integrates hardware sensing, edge AI processing, embedded vehicle control, network communication, backend services, database storage, and a user-facing web application into a single working prototype.

The system consists of three main system components: the robotic vehicle, the backend server, and the web dashboard. The robotic vehicle is built on a four-wheel chassis equipped with a Pi Camera V3, a Raspberry Pi 5, an Arduino Uno R3, an L298N motor driver, a DC motor, and a steering servo. The Raspberry Pi serves as the edge processing unit, running YOLO-based AI models to analyze the camera feed and generate movement commands. The Arduino receives these commands through Serial UART and drives the motor and steering servo accordingly.

The backend server runs a NestJS application that receives IoT event data from the Raspberry Pi, stores it in a SQLite database managed by Prisma ORM, and exposes REST API endpoints for the dashboard. The Next.js web dashboard provides a monitoring interface where authorized users can view device status, recent events, and alerts, and perform actions such as marking alerts as read or handled. This matches the project structure described in the project file, where the system is divided into the robotic vehicle, backend server, database, and web dashboard modules.

The data flow in the system follows a clear and linear path. The Pi Camera V3 captures video frames, which are processed by the Raspberry Pi using yolov8n.pt for person detection and best.pt for human state classification. Based on the detection results, the Raspberry Pi sends movement commands to the Arduino through Serial UART and reports structured event data to the NestJS backend through HTTP POST

requests over Wi-Fi. The backend stores the event, updates the device status, and generates an alert if the event is critical. The dashboard then retrieves and displays this information by polling the backend API every three seconds.

The architecture is designed with a clear separation of responsibilities between components. The Raspberry Pi handles AI inference and decision-making, the Arduino handles real-time motor control, the NestJS backend handles data management and business logic, and the Next.js dashboard handles user interaction and alert management. This modular design makes the system easier to develop, test, maintain, and extend in future versions.

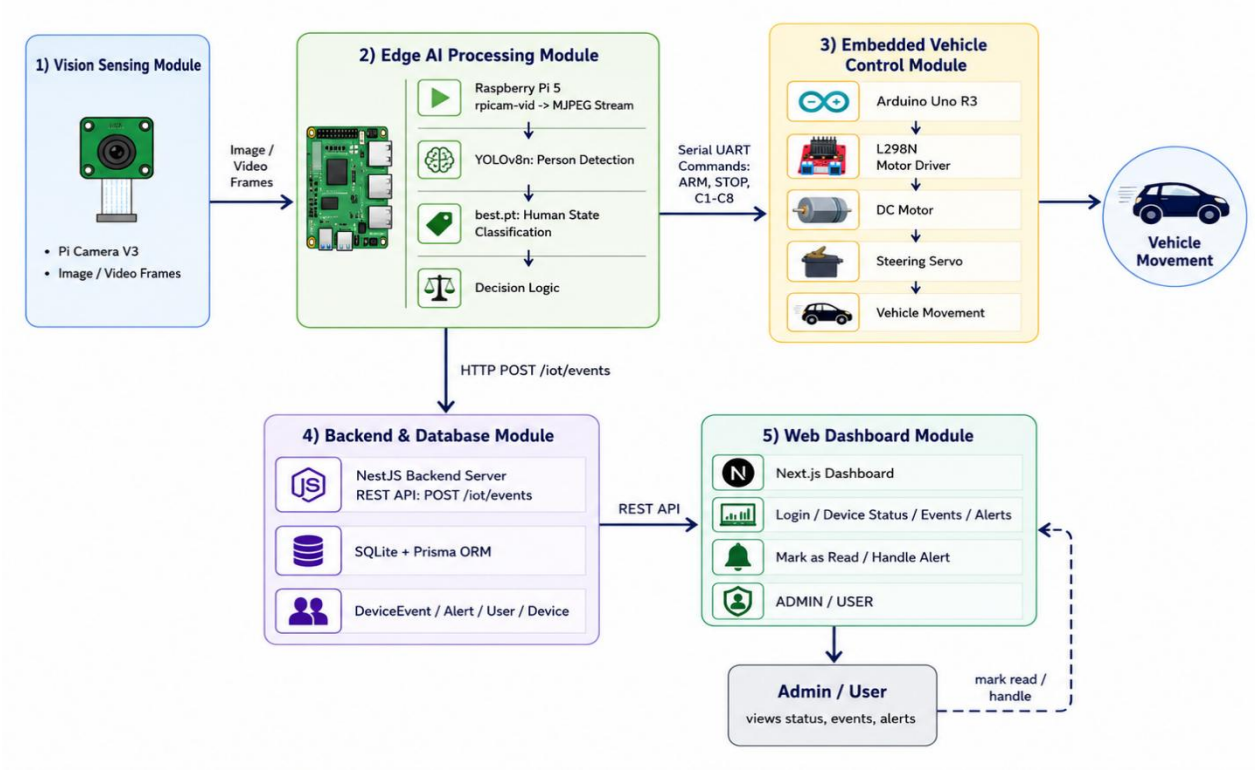


Figure 3.1. Overall implementation architecture of the proposed AI-IoT robotic car system

3.2 Experimental Environment and Testbed Setup

The experimental environment is designed to test the proposed AI-IoT robotic car in a outdoor environment. The purpose of this testbed is to verify whether the robot can capture visual data, detect and follow a person, classify human states, control vehicle movement, and send event data to the backend and dashboard. Since the system is

developed as a prototype, the experiment is conducted in a small-scale environment instead of a real healthcare facility or industrial site.

The testbed consists of a four-wheel robotic vehicle, a local Wi-Fi network, a backend server, and a web dashboard. The robotic vehicle carries the main hardware components, including the Pi Camera V3, Raspberry Pi 5, Arduino Uno R3, L298N motor driver, DC motor, steering servo, power supply, and vehicle chassis. The Pi Camera V3 is mounted at the front of the robot to capture video frames of the target person. The Raspberry Pi processes the video stream locally, while the Arduino controls the motor and steering system based on commands received from the Raspberry Pi.

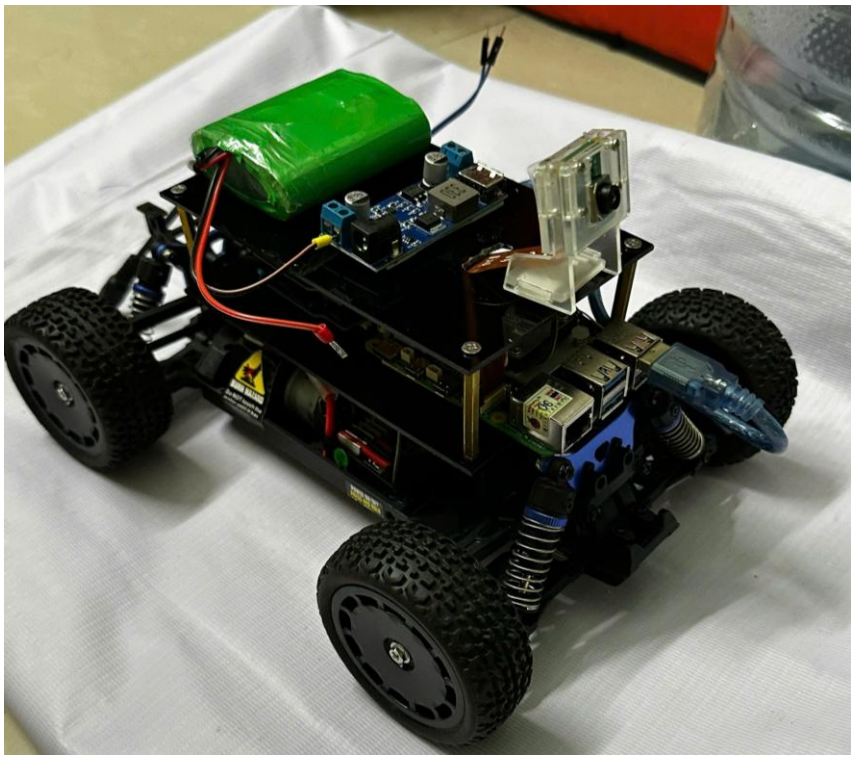


Figure 3.2. Experimental testbed setup of the proposed AI-IoT robotic car system.

The backend and dashboard are deployed in the same local network as the Raspberry Pi. The NestJS backend server receives event data from the robot through HTTP POST requests, stores the data in a SQLite database using Prisma ORM, and provides REST API endpoints for the dashboard. The Next.js dashboard is accessed through a web browser and is used to monitor device status, recent events, and alerts.

In this setup, the dashboard retrieves updated data from the backend by polling the API every three seconds.

The software environment includes Python-based AI processing on the Raspberry Pi, YOLO-based models for person detection and human state classification, Arduino firmware for embedded motor control, and web technologies for backend and dashboard development. The Raspberry Pi runs the camera and AI pipeline, extracts frames from the video stream, applies yolov8n.pt for person detection, applies best.pt for human state classification, and sends the corresponding commands or events to other system components.

The testbed is arranged so that the robot can move on a flat indoor surface while a person stands, sits, walks, or simulates a fall within the camera view. During testing, the system behavior is observed through both the physical movement of the robot and the information displayed on the dashboard. This setup allows the main functions of the system to be verified, including human-following, stopping behavior, person-lost handling, fall event reporting, backend event storage, and dashboard alert display.

Table 3.2. Experimental testbed setup of the proposed AI-IoT robotic car system.

Testbed Element	Component / Technology	Purpose
Robotic vehicle	Four-wheel chassis	Provides the physical platform for movement.
Vision sensor	Pi Camera V3	Captures image/video data from the environment.
Edge device	Raspberry Pi 5	Runs AI processing and decision logic.
Embedded controller	Arduino Uno R3	Controls motor and steering actuation.

Motor driver	L298N	Drives the DC motor based on Arduino signals.
Actuators	DC motor, steering servo	Provide vehicle movement and steering.
Network	Local Wi-Fi	Connects Raspberry Pi, backend server, and dashboard.
Backend server	NestJS	Receives events and manages backend logic.
Database	SQLite + Prisma ORM	Stores users, devices, events, and alerts.
Dashboard	Next.js web application	Displays device status, events, and alerts.

3.3 Hardware Design of the Robotic Vehicle

The robotic vehicle is the main physical platform of the proposed AI-IoT system. It is designed to carry the sensing, processing, and control components needed for human-following and fall detection. The vehicle is built on a four-wheel chassis, which provides a stable base for indoor testing and allows the robot to move forward, turn left, turn right, search for the target person, and stop when necessary.

The visual sensing function is provided by the Pi Camera V3, which is mounted at the front of the vehicle. This position allows the camera to capture the area in front of the robot and observe the target person during operation. The camera sends image and video data to the Raspberry Pi 5, which is installed on the vehicle as the main edge processing device. The Raspberry Pi receives the camera stream, runs YOLO-based AI models, and makes movement or alert decisions based on the detected person position and human state.

For embedded vehicle control, the system uses an Arduino Uno R3 as the low-level controller. The Raspberry Pi sends movement commands to the Arduino through a USB Serial UART connection. After receiving a command, the Arduino generates control signals for the motor and steering system. This design separates the AI processing task from the real-time actuator control task, making the hardware system more stable and easier to debug.

The movement system consists of an L298N motor driver, a DC motor, and a steering servo. The L298N motor driver receives control signals from the Arduino and drives the DC motor by adjusting its direction and speed. The steering servo is controlled separately using PWM signals to adjust the vehicle direction. With this configuration, the robot can follow a person based on the camera result and stop immediately when a fall or unsafe state is detected.

The hardware layout is arranged to keep the wiring simple and the vehicle balanced. The Raspberry Pi and Arduino are placed on the chassis so that they can be connected easily through USB Serial. The L298N motor driver is positioned close to the motor to reduce wiring complexity. The camera is mounted at the front to provide a clear field of view, while the power supply is arranged to provide enough energy for the processing boards, motor driver, and actuators.

Table 3.3. Main hardware components of the robotic vehicle

Hardware Component	**Role in the System**
Four-wheel chassis	Provides the mechanical structure and movement platform.
Pi Camera V3	Captures image and video data from the environment.
Raspberry Pi 5	Runs edge AI processing, YOLO models, and decision logic.

Arduino Uno R3	Receives commands from Raspberry Pi and controls actuators.
L298N motor driver	Drives the DC motor and controls motor direction/speed.
DC motor	Provides the main driving force for vehicle movement.
Steering servo	Controls the turning direction of the vehicle.
Power supply	Provides electrical power for the boards, motor driver, and actuators.

Overall, the robotic vehicle hardware is designed to support the complete AI-IoT workflow. The Pi Camera V3 provides visual input, the Raspberry Pi 5 performs AI-based perception and decision-making, the Arduino Uno R3 manages low-level control, and the motor driver, DC motor, and steering servo convert system commands into physical movement.

3.4 Edge AI Software Design on Raspberry Pi

The Raspberry Pi 5 is used as the main edge computing device in the proposed system. Its role is to receive video data from the Pi Camera V3, process the frames locally using AI models, determine the person's position and state, and generate corresponding control commands or IoT events. By running the AI pipeline directly on the Raspberry Pi, the system can reduce network traffic and respond faster to movement or fall-related situations.

The camera stream is captured using the Pi Camera V3 and processed on the Raspberry Pi. In the implementation, the camera output is read as an MJPEG stream generated by rpicam-vid. The Python program extracts frames from this stream and uses the latest available frame for AI inference. This helps reduce delay because older frames can be discarded instead of being processed one by one when the system is busy.

The AI processing pipeline uses two YOLO-based models. The first model, yolov8n.pt, is used for person detection. It detects whether a person appears in the camera frame and returns the bounding box position of the detected person. From this bounding box, the system calculates the horizontal center of the person and divides the frame into left, center, and right regions. This information is then used to decide whether the vehicle should move forward, turn left, turn right, or search for the person.

The second model, best.pt, is used for human state classification. This model classifies the detected person into three states: standing, sitting, and fallen. If the person is standing, the robot continues following based on the detected position. If the person is sitting or fallen, the system sends a stop command to the Arduino for safety. When the state is classified as fallen, the Raspberry Pi also sends a FALL_DETECTED event to the backend server. The project document describes this AI processing flow with yolov8n.pt for person detection and best.pt for standing, sitting, and fallen classification.

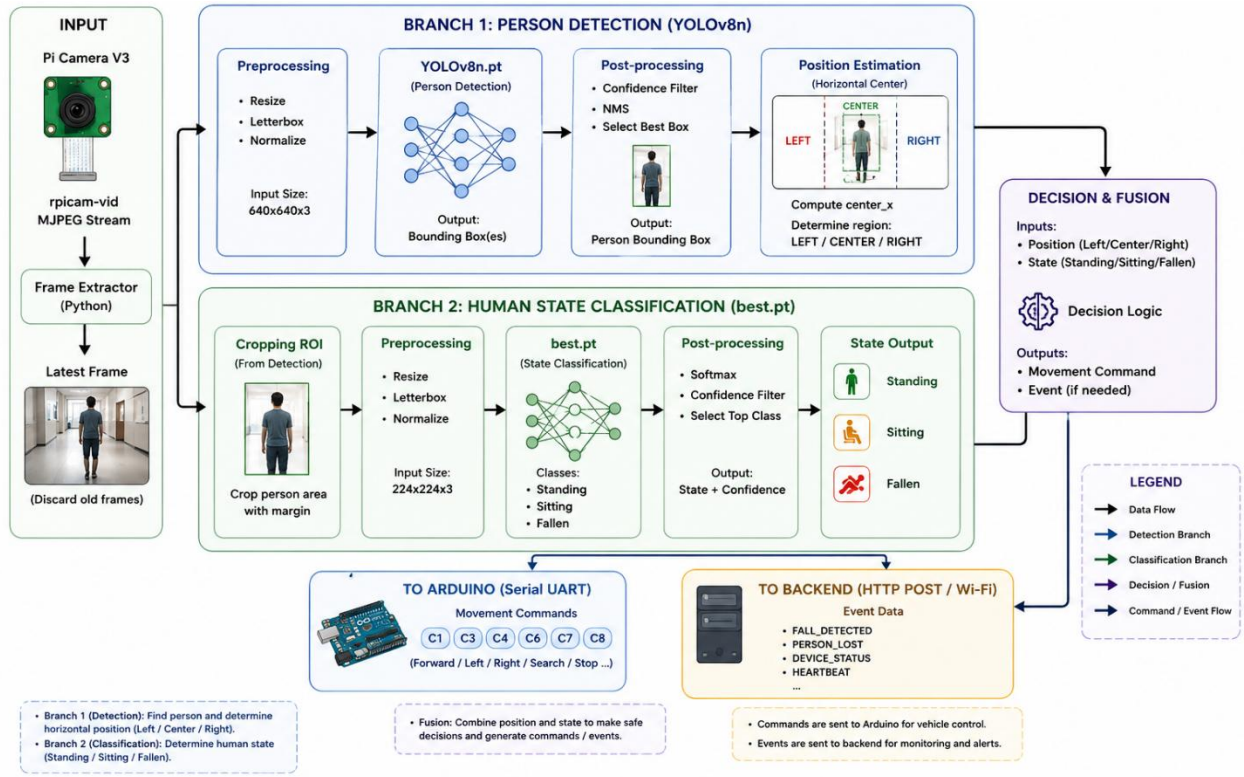


Figure 3.4. Edge AI software pipeline on Raspberry Pi for person detection and human state classification

To improve real-time performance, several optimization techniques are applied. The program reads frames continuously and keeps only the newest frame to avoid processing outdated images. The input image size for YOLO inference can be reduced to lower the computational load. In addition, the human state classification model does not need to run on every frame because posture changes are usually slower than position changes. A smoothing method is also applied to the detected center position to reduce unstable commands caused by small detection noise.

After AI processing, the Raspberry Pi performs two main actions. First, it sends movement commands such as C1, C3, C4, C6, C7, or C8 to the Arduino through Serial UART/USB. Second, it sends structured IoT event data to the NestJS backend through HTTP POST requests when important states or alerts need to be recorded. Therefore, the Raspberry Pi acts as the bridge between the camera, AI models, vehicle control, and backend monitoring system.

Table 3.4. Edge AI software functions on Raspberry Pi

Software Function	Implementation	Role in the System
Camera streaming	Pi Camera V3 with rpicam-vid	Provides video input for AI processing.
Frame extraction	Python MJPEG frame reading	Extracts the latest frame for inference.
Person detection	yolov8n.pt	Detects the person and bounding box position.
Position estimation	Bounding box center calculation	Determines left, center, or right region.
State classification	best.pt	Classifies standing, sitting, or fallen state.
Decision logic	Python control logic	Generates vehicle commands and IoT events.

Arduino communication	Serial UART/USB	Sends movement commands to Arduino.
Backend communication	HTTP POST /iot/events	Sends event data to the NestJS backend.

Overall, the edge AI software on the Raspberry Pi is the core intelligence of the robotic car. It allows the system to understand visual information, follow a person, detect abnormal human states, control the vehicle, and report important events to the monitoring system.

3.5 Backend Server and Database Design

The backend server is responsible for receiving IoT event data from the Raspberry Pi, processing system logic, storing data in the database, and providing API endpoints for the web dashboard. In this project, the backend is implemented using NestJS, a Node.js framework that supports modular server-side application development. NestJS is suitable for this system because it allows the backend to be organized into separate modules such as authentication, device management, IoT event handling, alert management, and database access.

The backend acts as the middleware layer between the robotic vehicle and the web dashboard. When the Raspberry Pi detects a human state or an important event, it sends a structured JSON payload to the backend through an HTTP POST request. The main endpoint used for receiving IoT events is /iot/events. After receiving the event, the backend validates the data, stores it as a device event, updates the current device status, and creates an alert if the event is critical, such as FALL_DETECTED or PERSON_LOST.

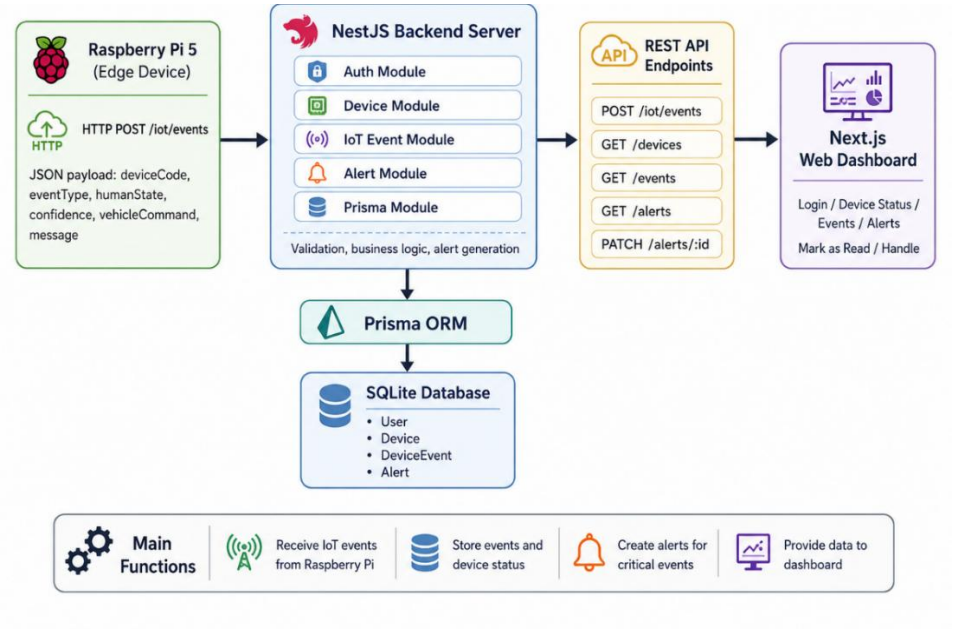


Figure 3.5. Backend server and database architecture of the proposed system

The database is implemented using SQLite and managed through Prisma ORM. SQLite is suitable for this prototype because it is lightweight, simple to configure, and does not require a separate database server. Prisma ORM is used to define database models and simplify database operations in the backend code. Instead of writing raw SQL queries directly, the backend can use Prisma Client to create, read, update, and query records in a structured way.

The main database entities in this project include User, Device, DeviceEvent, and Alert. The User table stores user account information and role data. The Device table stores information about the robotic car, such as its device code and current status. The DeviceEvent table stores events sent from the Raspberry Pi, including event type, human state, confidence score, vehicle command, and message. The Alert table stores warning records generated from critical events and tracks their lifecycle, such as NEW, READ, and HANDLED.

The backend also exposes REST API endpoints for the dashboard. The dashboard uses these APIs to retrieve device status, recent events, and alert information. Users can also update alert status through the dashboard, such as marking an alert as read or handled. This design separates the frontend interface from the backend logic, making the system easier to maintain and extend.

Table 3.5. Main backend modules and functions

Backend Module	Main Function
Auth module	Handles user login, authentication, and token generation.
Device module	Manages robotic vehicle information and current device status.
IoT Event module	Receives and stores event data from the Raspberry Pi.
Alert module	Creates, displays, and updates alert records.
Prisma module	Provides database connection and data access through Prisma ORM.

Table 3.6. Main database entities in the proposed system

Database Entity	Stored Information	Role in the System
User	Username, password hash, role	Manages dashboard users and permissions.
Device	Device code, name, status	Represents the robotic vehicle in the system.
DeviceEvent	Event type, human state, confidence, vehicle command, message	Stores events reported by the Raspberry Pi.
Alert	Alert type, status, related device/event	Stores critical warnings and alert lifecycle.

Overall, the backend server and database design provide the data management foundation of the system. The NestJS server receives IoT events, applies business

logic, stores records through Prisma ORM, and provides structured data to the Next.js dashboard. This allows the robotic car system to record its operating history, generate alerts, and support user monitoring through a web interface.

3.6 IoT Data Pipeline and Control Flow

The IoT data pipeline describes how data moves through the proposed system from the physical environment to the monitoring dashboard. In this project, the pipeline starts from visual data captured by the Pi Camera V3 and ends with vehicle movement, event storage, and alert display on the web dashboard. This pipeline connects the sensing layer, edge AI processing, embedded control, backend service, database, and user interface into one complete workflow.

At the beginning of the pipeline, the Pi Camera V3 captures image and video data from the environment in front of the robotic vehicle. The video stream is processed by the Raspberry Pi 5, which extracts frames and applies YOLO-based models. The yolov8n.pt model detects the presence and position of a person, while the best.pt model classifies the human state as standing, sitting, or fallen.

After AI inference, the Raspberry Pi performs decision-making based on two main outputs: the person's position and the human state. If the person is standing and remains within the camera view, the Raspberry Pi generates a movement command such as moving forward, turning left, or turning right. This command is sent to the Arduino Uno R3 through Serial UART/USB communication. The Arduino then controls the L298N motor driver, DC motor, and steering servo to move the vehicle accordingly.

If an abnormal event is detected, such as a fallen state or the target person being lost from view, the control flow changes to safety mode. For a fallen state, the Raspberry Pi sends a stop command to the Arduino to stop the vehicle. At the same time, the Raspberry Pi prepares a structured IoT event and sends it to the NestJS backend server through an HTTP POST request. The event data may include fields

such as deviceCode, eventType, humanState, confidence, vehicleCommand, and message.

On the server side, the backend receives the event through the /iot/events endpoint. The server validates the received data, stores the event in the SQLite database through Prisma ORM, updates the current device status, and creates an alert if the event is critical. The Next.js dashboard retrieves updated information from the backend through REST API requests. In the current implementation, the dashboard polls the backend every three seconds to display the latest device status, recent events, and alerts.

This data pipeline shows how the system connects real-world sensing with digital monitoring and control. The camera provides visual input, the Raspberry Pi converts the visual data into AI-based decisions, the Arduino converts commands into physical movement, the backend stores and processes events, and the dashboard presents the system state to users. As a result, the system can perform human-following, fall detection, event reporting, and alert monitoring in one integrated IoT workflow.

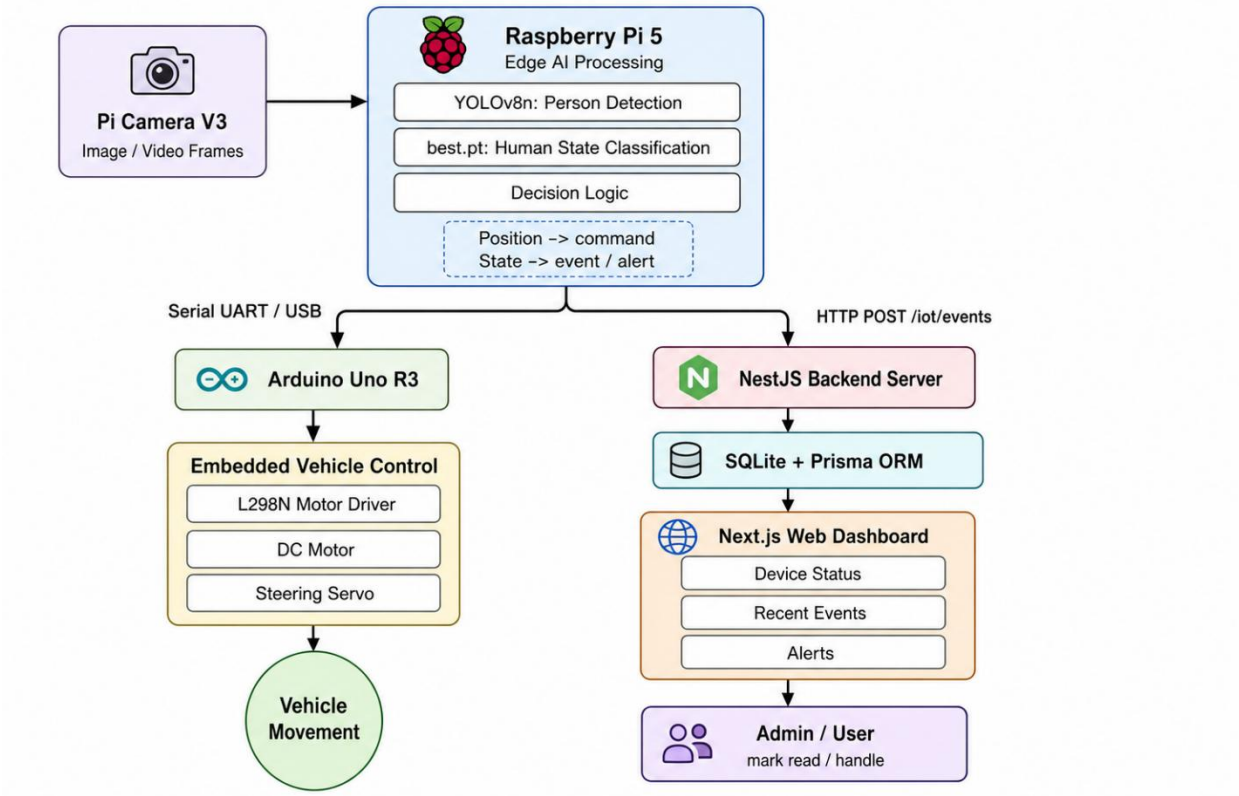


Figure 3.6. End-to-end IoT data pipeline and control flow of the proposed system

Table 3.7. Main data flow and control actions in the proposed system

Stage	Input	Processing / Action	Output
Camera sensing	Real-world scene	Pi Camera V3 captures video frames	Image/video stream
Edge AI processing	Camera frames	YOLOv8n detects person, best.pt classifies state	Position and human state
Decision logic	Position and state	Determines movement command or alert event	Vehicle command / IoT event
Embedded control	Serial command	Arduino controls motor driver and servo	Vehicle movement or stop
Backend processing	JSON event	NestJS stores event and creates alert if needed	DeviceEvent / Alert record
Dashboard monitoring	Backend API data	Next.js displays status, events, and alerts	User monitoring and alert handling

3.7 Web Dashboard Design and User Interface

The web dashboard is the user-facing interface of the proposed AI-IoT robotic car system. It allows users to monitor the robotic vehicle, view recent events, check alerts, and manage safety-related information through a web browser. Instead of directly accessing the Raspberry Pi or backend server, users can interact with the system through a clear and organized dashboard interface.

The dashboard is developed using Next.js, which provides a modern framework for building web applications. It communicates with the NestJS backend server through REST API requests. The backend provides data such as device status, recent IoT events, and alert records, while the dashboard displays this information in a readable format for users.

The main functions of the dashboard include user login, device monitoring, event history display, and alert management. After logging in, users can view the current

status of the robotic vehicle, such as whether it is following a person, stopped, searching, or reporting an abnormal event. The event section shows recent records sent from the Raspberry Pi, including human state, confidence score, vehicle command, and event message. The alert section displays important warnings such as FALL_DETECTED or PERSON_LOST.

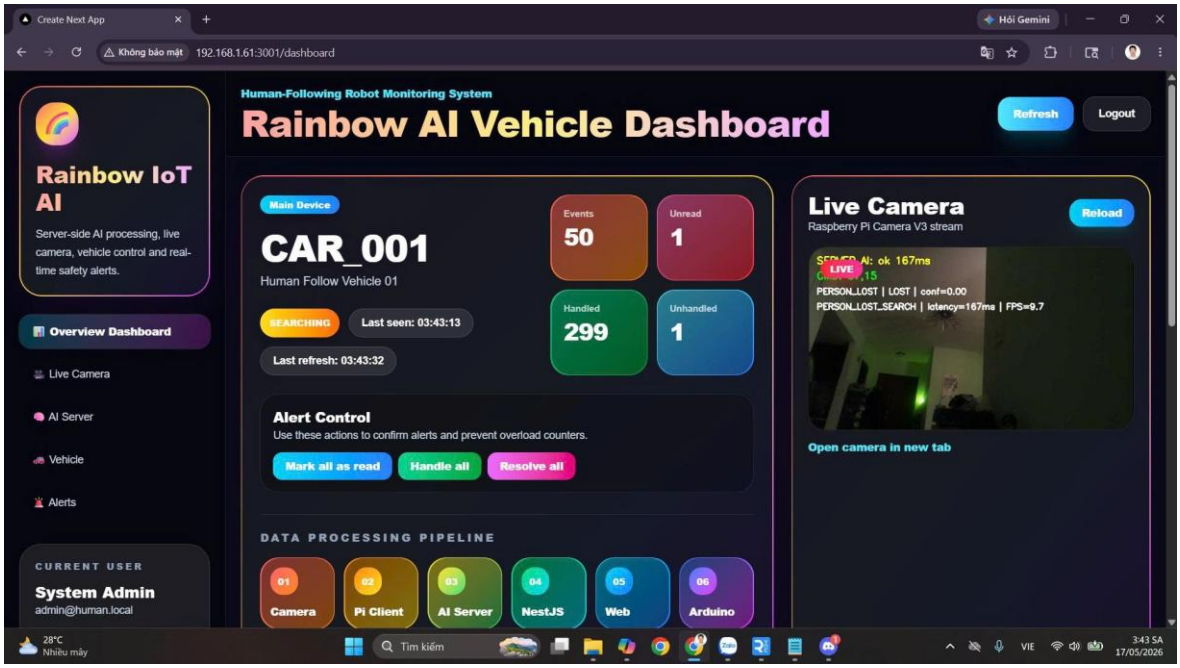


Figure 3.7. Web dashboard interface for monitoring device status, events, and alerts

For alert management, the dashboard allows users to update the status of an alert. When a critical event is first generated, the alert is shown as NEW. After the user views the alert, it can be marked as READ. When the situation has been checked or resolved, the alert can be marked as HANDLED. This alert lifecycle helps users track safety-related events and prevents important warnings from being ignored.

The dashboard also supports role-based access control. In this project, there are two main roles: ADMIN and USER. An ADMIN account can view and manage all devices, events, and alerts in the system. A USER account can only access the devices and alerts assigned to that user. This design helps protect monitoring data and ensures that each user can only interact with permitted system information.

In the current implementation, the dashboard updates its data by polling the backend API every three seconds. This approach is simple and suitable for the prototype because it allows the dashboard to refresh device status, events, and alerts without requiring complex real-time infrastructure. In future versions, this mechanism can be improved using WebSocket or MQTT to support faster real-time updates.

Table 3.8. Main dashboard functions in the proposed system.

Dashboard Function	Description	Purpose
Login	Allows users to access the dashboard using credentials.	Protects the monitoring interface.
Device status view	Displays the current status of the robotic vehicle.	Helps users monitor system operation.
Recent events	Shows event records sent from the Raspberry Pi.	Provides history of detection and vehicle behavior.
Alert list	Displays critical warnings such as fall or person-lost events.	Helps users identify abnormal situations.
Mark as read	Changes alert status from NEW to READ.	Confirms that the alert has been viewed.
Handle alert	Changes alert status to HANDLED.	Indicates that the issue has been checked or resolved.
Role-based access	Separates ADMIN and USER permissions.	Controls access to device and alert data.
Polling update	Refreshes dashboard data every three seconds.	Provides near real-time monitoring.

Overall, the web dashboard provides the main interaction layer between users and the AI-IoT robotic car system. It transforms backend data into a practical monitoring

interface, allowing users to observe device status, review event history, and respond to important alerts. This makes the system easier to use, demonstrate, and manage during testing.

3.8 Security Implementation

Security implementation is necessary because the proposed system contains user accounts, device information, event history, and safety alerts. Without proper security mechanisms, unauthorized users may access monitoring data, modify alert status, or view information from devices that they are not allowed to manage. Therefore, the system applies basic security features at the backend and dashboard levels to protect user access and API communication.

In this project, user authentication is implemented using JWT-based authentication. When a user logs in with valid credentials, the NestJS backend verifies the account information and generates a JSON Web Token, also known as JWT. This token is then stored on the client side and attached to later API requests. When the dashboard sends a request to protected API endpoints, the backend checks the token before allowing access to device data, event history, or alert information.

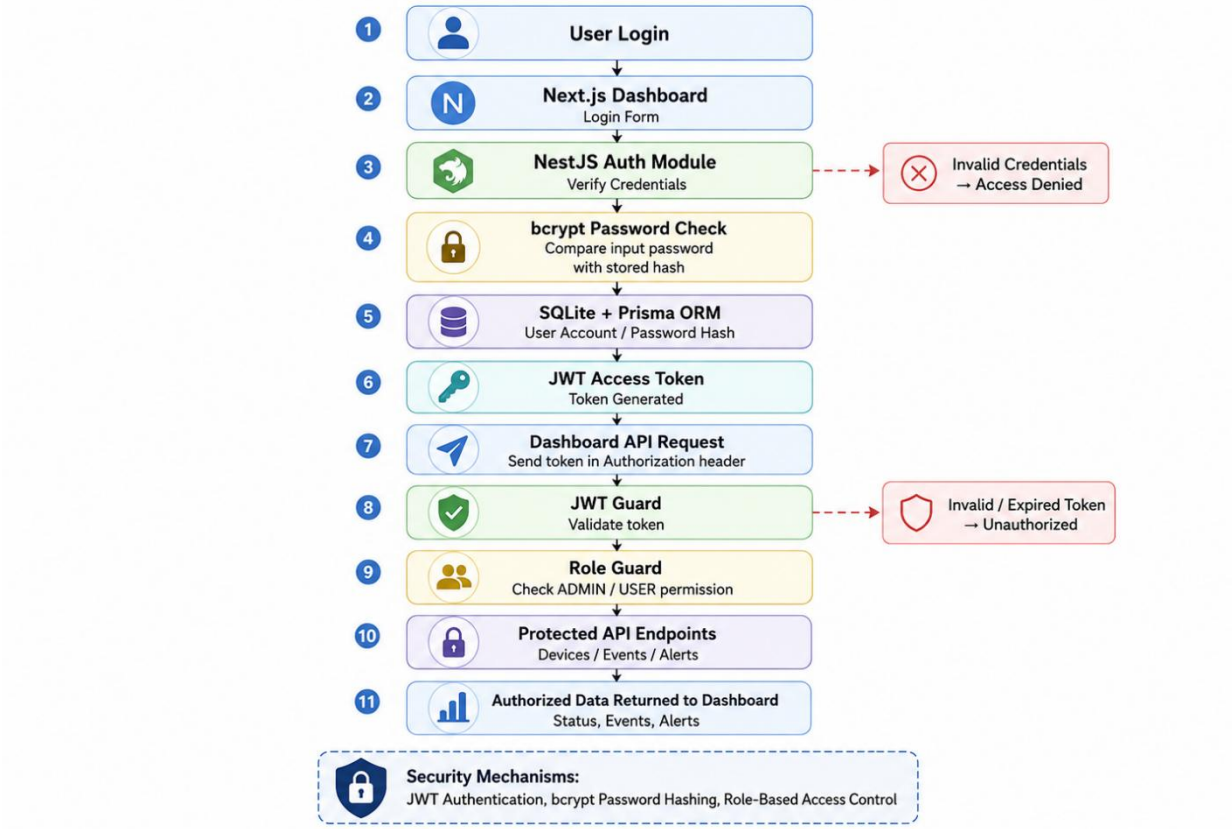


Figure 3.8. Authentication and authorization flow in the proposed system

Password protection is implemented using bcrypt password hashing. Instead of storing raw passwords in the database, the backend stores hashed password values. When a user logs in, the password entered by the user is hashed and compared with the stored hash. This improves account security because the original password is not directly stored in the database.

The system also applies role-based access control to separate user permissions. Two main roles are used in the system: ADMIN and USER. An ADMIN account can view and manage all devices, events, and alerts. A USER account can only access assigned devices and related alert information. This role separation helps prevent unauthorized access and makes the system more suitable for multi-user monitoring.

In the NestJS backend, authorization guards are used to protect API endpoints. These guards check whether the request contains a valid JWT token and whether the user role has permission to access the requested resource. For example, endpoints

related to alert management or device monitoring should only return data to authenticated users with suitable permissions. This ensures that the dashboard can only display data that the current user is allowed to view.

The dashboard also supports the security workflow by requiring users to log in before accessing the monitoring interface. After login, the dashboard uses the access token when requesting device status, recent events, and alerts from the backend. If the token is missing, expired, or invalid, the backend rejects the request and the user must log in again.

Table 3.9. Security mechanisms implemented in the proposed system

Security Mechanism	Implementation	Role in the System
JWT authentication	Access token generated after login	Verifies user identity for protected API requests.
bcrypt password hashing	Passwords stored as hashed values	Prevents storing raw passwords in the database.
Role-based access control	ADMIN and USER roles	Limits data access based on user permission.
Authorization guard	NestJS backend guard	Protects API endpoints from unauthorized requests.
Protected dashboard access	Login required before monitoring	Prevents unauthenticated users from viewing system data.

Overall, the security implementation provides a basic protection layer for the AI-IoT robotic car system. JWT authentication verifies users, bcrypt protects account

passwords, and role-based access control limits what each user can access. Although the current system is developed as a prototype, these mechanisms create a foundation for safer deployment and can be extended in the future with HTTPS, refresh tokens, device authentication, audit logs, and rate limiting.

3.9 Safety Response and Alert Generation Logic

The safety response and alert generation logic is one of the most important parts of the proposed AI-IoT robotic car system. Since the robot is designed for human-following and fall detection, it must not only recognize human states but also respond properly when an abnormal situation occurs. The main purpose of this logic is to ensure that the vehicle reacts safely at the local control level and that important events are reported to the backend and dashboard for user monitoring.

The safety logic starts from the AI inference result on the Raspberry Pi. After each processed frame, the system receives two main outputs: the detected person position and the classified human state. The position output is used for movement control, while the human state output is used for safety decision-making. If the person is classified as standing, the robot continues following the person based on their position in the camera frame. If the person is classified as sitting or fallen, the robot stops to avoid unsafe movement.

When a fallen state is detected with sufficient confidence, the Raspberry Pi immediately sends the C8 stop command to the Arduino Uno R3 through Serial UART. The Arduino then stops the vehicle by controlling the motor driver and steering system. This local stop action is performed before the event is sent to the backend, so the robot can respond even if the network connection is temporarily unstable. After stopping the vehicle, the Raspberry Pi sends a structured FALL_DETECTED event to the NestJS backend through an HTTP POST request.

The system also handles cases where the target person is lost from the camera view. If no person is detected for a short period, the robot may enter a searching behavior by turning slowly toward the last known direction. If the person is still not

found after a timeout, the system can stop the robot and send a PERSON_LOST event to the backend. This helps prevent the robot from moving blindly when the target is no longer visible.

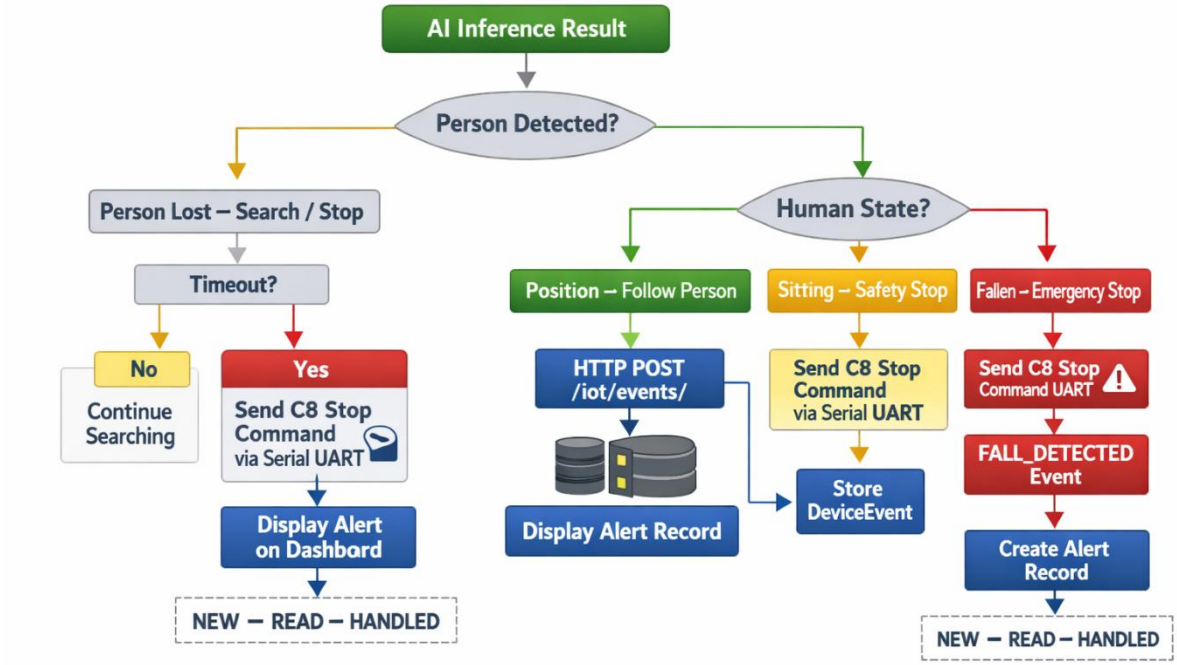


Figure 3.9. Safety response and alert generation flow of the proposed system

On the backend side, the NestJS server receives the event through the /iot/events endpoint. The server stores the event as a DeviceEvent record, updates the current device status, and checks whether the event type requires an alert. For critical events such as FALL_DETECTED and PERSON_LOST, the backend automatically creates an Alert record. Each alert is initialized with the NEW status and can later be updated by the user through the dashboard.

The alert lifecycle contains three main states: NEW, READ, and HANDLED. A NEW alert means that a critical event has just been generated and has not been viewed by the user. After the user opens or checks the alert on the dashboard, it can be marked as READ. When the situation has been checked or resolved, the alert can be marked as HANDLED. This lifecycle allows the system to track the status of each safety-related event in an organized way.

Table 3.10. Safety response and alert generation logic

Condition / Event	Local Vehicle Response	Backend / Dashboard Response
Standing	Continue following based on person position.	Store normal event/status if needed.
Sitting	Stop the vehicle for safety.	Store event for monitoring.
Fallen	Send C8 stop command immediately.	Create FALL_DETECTED alert.
Person lost	Search based on last known direction, then stop after timeout.	Create PERSON_LOST event/alert.
NEW alert	—	Alert is created and displayed on dashboard.
READ alert	—	User has viewed the alert.
HANDLED alert	—	User has checked or resolved the situation.

Overall, the safety response and alert generation logic ensures that abnormal situations are handled at both the vehicle level and the monitoring system level. The Raspberry Pi and Arduino provide immediate local response, while the backend and dashboard provide event recording, alert tracking, and user interaction. This design improves the reliability and safety of the human-following robot during testing and demonstration.

3.10 Algorithm-to-Hardware Mapping

Algorithm-to-hardware mapping describes how each software function or algorithm in the system is assigned to a specific hardware or software component. This mapping is important because the proposed system includes many connected parts, such as the camera, Raspberry Pi, Arduino, motor driver, backend server, database, and web dashboard. A clear mapping helps explain how the complete AI-IoT workflow is implemented in practice.

In this project, the AI-related tasks are mainly handled by the Raspberry Pi 5. The Raspberry Pi receives video frames from the Pi Camera V3, runs yolov8n.pt for person detection, runs best.pt for human state classification, and applies decision logic to generate movement commands or event data. Since these tasks require image processing and AI inference, they are assigned to the Raspberry Pi instead of the Arduino.

The low-level vehicle control tasks are handled by the Arduino Uno R3. After receiving commands from the Raspberry Pi through Serial UART, the Arduino generates control signals for the L298N motor driver and the steering servo. This allows the vehicle to move forward, turn left, turn right, search for the person, or stop when an unsafe condition is detected. This separation makes the system more stable because AI processing and motor control are handled by different devices.

The backend and monitoring functions are implemented on the server and dashboard side. The NestJS backend receives event data from the Raspberry Pi, stores device events, updates device status, and creates alerts when critical events occur. The SQLite database managed by Prisma ORM stores users, devices, events, and alerts. The Next.js dashboard displays the system status and allows users to view and handle alerts.

Table 3.11. Algorithm-to-hardware mapping in the proposed AI-IoT robotic car system

Algorithm / Function	Assigned Component	Role in the System
Image acquisition	Pi Camera V3	Captures image and video data from the environment.
Frame extraction	Raspberry Pi 5	Extracts frames from the camera stream for AI processing.
Person detection	Raspberry Pi 5 + yolov8n.pt	Detects the person and bounding box in the camera frame.
Position estimation	Raspberry Pi 5	Calculates the person's position as left, center, or right.
Human state classification	Raspberry Pi 5 + best.pt	Classifies the person as standing, sitting, or fallen.
Movement decision logic	Raspberry Pi 5	Converts AI results into movement commands or event data.
Serial command transmission	Raspberry Pi 5 → Arduino Uno R3	Sends commands such as C1, C3, C4, C6, C7, or C8.
Motor control	Arduino Uno R3 + L298N	Controls DC motor direction and speed.
Steering control	Arduino Uno R3 + steering servo	Controls the turning direction of the vehicle.
Safety stop	Raspberry Pi 5 + Arduino Uno R3	Stops the vehicle when sitting, fallen, or unsafe states are detected.

Event reporting	Raspberry Pi 5 → NestJS backend	Sends IoT events through HTTP POST requests.
Alert generation	NestJS backend	Creates alerts for FALL_DETECTED or PERSON_LOST.
Data storage	SQLite + Prisma ORM	Stores users, devices, events, and alerts.
Dashboard monitoring	Next.js dashboard	Displays device status, event history, and alerts to users.
Access control	NestJS backend + Next.js dashboard	Supports JWT authentication and ADMIN/USER permissions.

Overall, the algorithm-to-hardware mapping shows how the system distributes tasks across different components. The Pi Camera V3 provides visual input, the Raspberry Pi 5 performs AI inference and decision-making, the Arduino Uno R3 controls the vehicle actuators, the NestJS backend manages events and alerts, and the Next.js dashboard provides the user interface. This mapping confirms that the proposed system is implemented as a complete end-to-end AI-IoT robotic car prototype.

CHAPTER 4. SYSTEM RESULTS AND EVALUATION

4.1 Experimental Setup

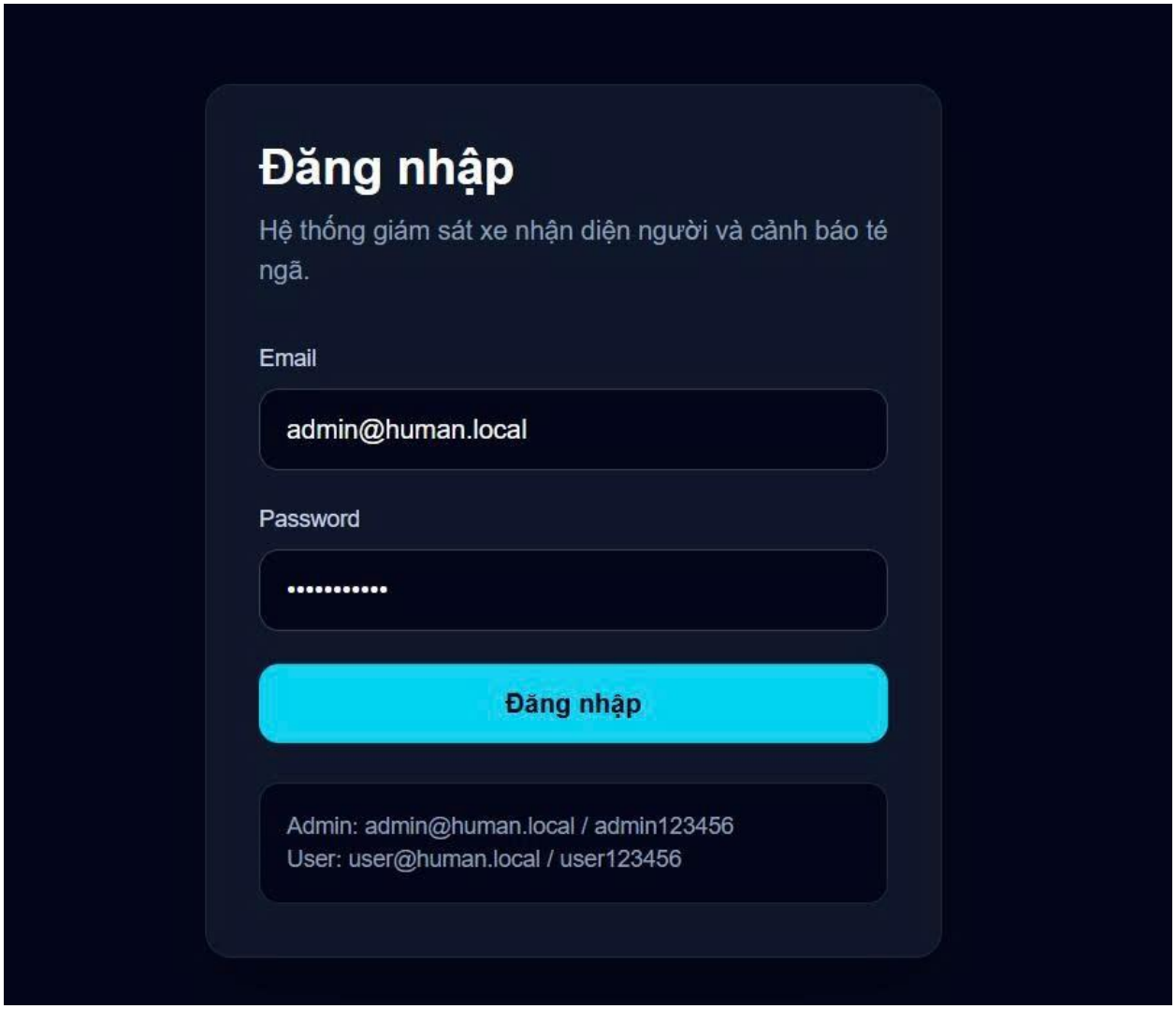


Figure 4.1. Log in to the dashboard

The login interface requires users to authenticate with an email and password before accessing the monitoring dashboard. The system supports two account roles: ADMIN and USER, each with different levels of access to device data and alert management.



Figure 4.2. Experimental setup for testing the proposed AI-IoT robotic car system

The experiment is conducted in an open outdoor environment. The robotic vehicle is positioned behind the target person, with the Pi Camera V3 facing forward to capture the person within the camera frame. This setup allows the robot to detect, follow, and monitor the person during real-world testing.

4.2 Testing Scenarios

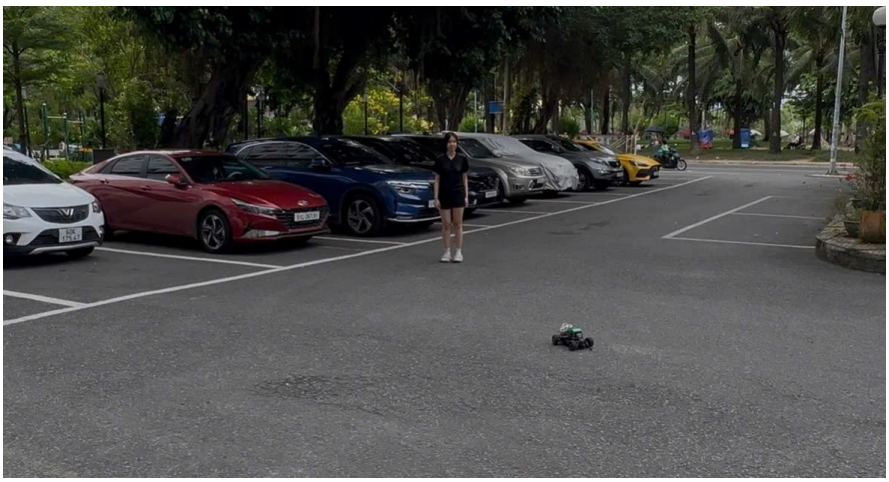


Figure 4.3. Main testing scenarios of the proposed system when person turns left

This scenario tests the vehicle's ability to follow a person who moves to the left side of the camera frame. When the person's detected position shifts to the left zone, the

Raspberry Pi sends a turn-left command to the Arduino, causing the vehicle to adjust its direction accordingly.



Figure 4.4. Main testing scenarios of the proposed system when person falls

This scenario simulates a fall event in which the person collapses to the ground. The best.pt model classifies the human state as fallen, triggering an immediate C8 stop command to the Arduino and sending a FALL_DETECTED event to the backend server to generate a critical alert.

4.3 Results of the Edge AI Processing Pipeline

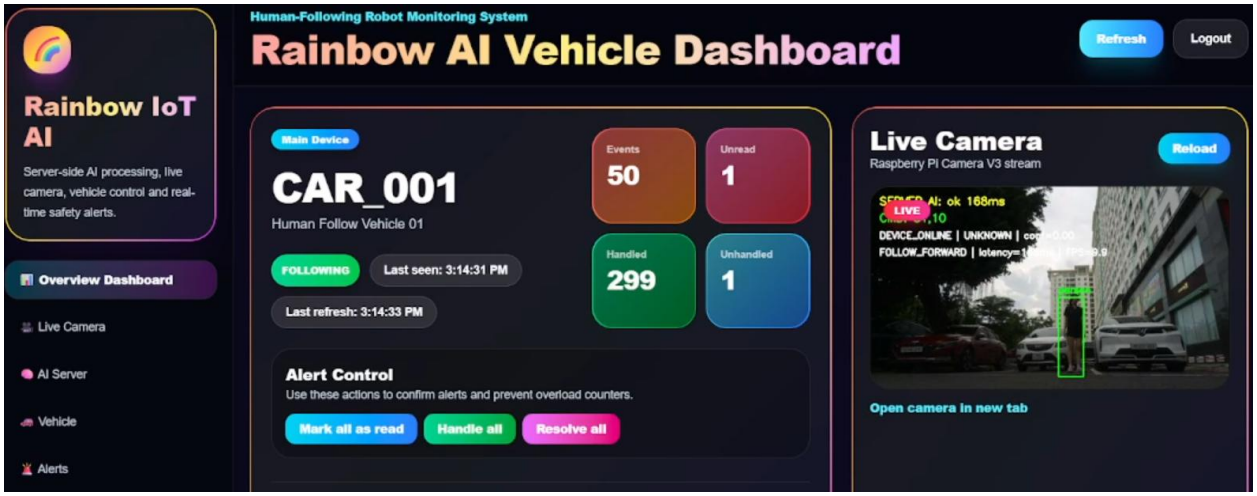


Figure 4.5. Application interface

The web dashboard displays the main device CAR_001 with its current status as FOLLOWING, along with the total number of events, unread alerts, and handled

alerts. The Live Camera panel on the right shows the real-time video stream from the Raspberry Pi, with AI detection overlay indicating the detected person and current system state.

4.4 Fall Detection and Alert Generation Results

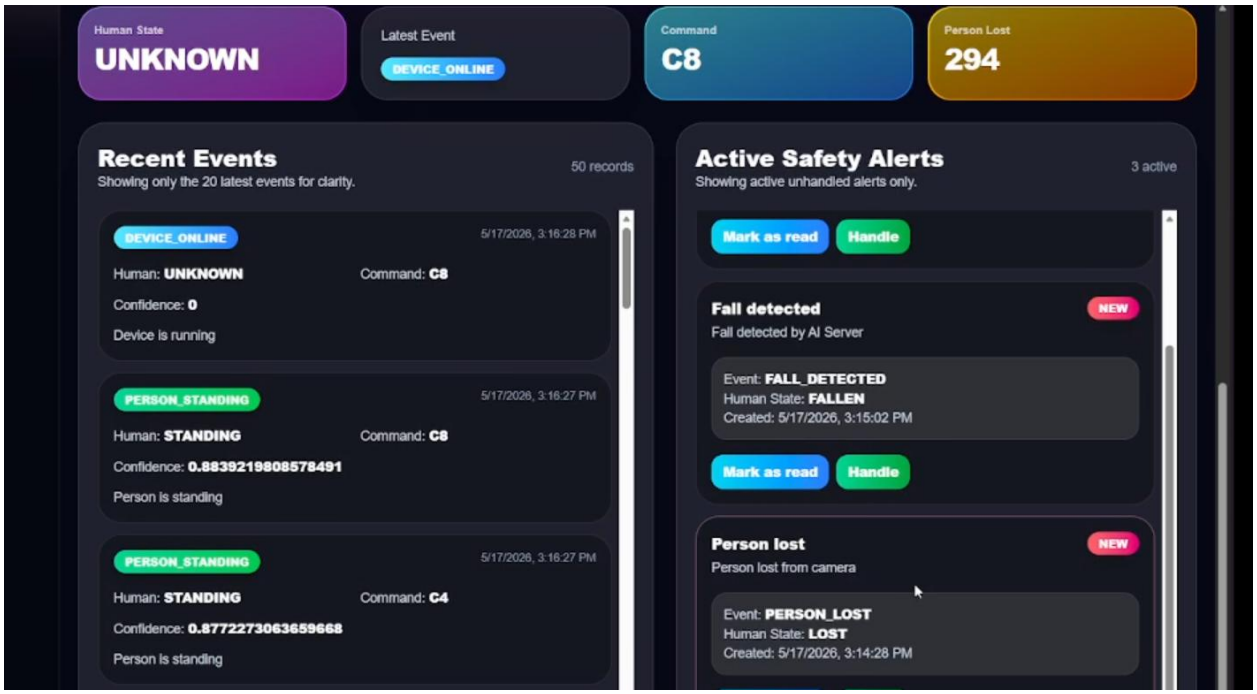


Figure 4.6. Fall detection and alert generation workflow

When a fall is detected, the dashboard immediately displays a FALL_DETECTED alert in the Active Safety Alerts panel with the NEW status. The Recent Events panel also records the corresponding PERSON_STANDING and state transition events with their confidence scores and timestamps, confirming that the full alert pipeline from AI inference to dashboard display is working correctly.

4.5 System Performance Evaluation

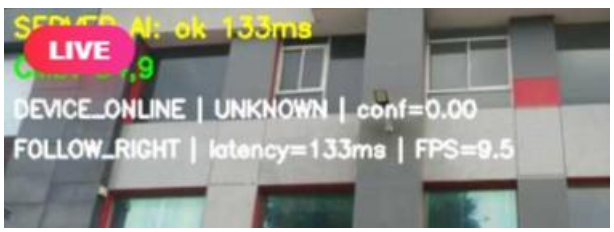


Table 4.7. System performance evaluation metrics

The live camera overlay shows key performance indicators during system operation, including an AI server response time of 133ms, a current FPS of 9.5, and a control latency of 133ms. These metrics confirm that the edge AI pipeline on the Raspberry Pi can process frames and generate vehicle commands at a near real-time rate sufficient for human-following and fall detection tasks.

4.6 Discussion, Limitations, and Improvement Directions

The results from the testing phase show that the proposed system can perform its main functions in a real outdoor environment. The vehicle successfully detected and followed a person, classified human states including standing, sitting, and fallen, and reported events to the backend server. The dashboard correctly displayed device status, recent events, and safety alerts, while alert management functions such as Mark as Read and Handle worked as expected. The system achieved an average FPS of 9.5 and a control latency of approximately 133ms, which is sufficient for smooth human-following behavior under normal conditions.

However, the system has several limitations. The detection accuracy of best.pt depends on camera angle, viewing distance, and lighting conditions, and may decrease when the person is partially occluded or in low-light environments. The dashboard polling interval of three seconds introduces a delay between event detection and alert display, and repeated PERSON_LOST events can generate excessive unhandled alerts. The system also currently supports only single-person tracking.

For future development, the best.pt model should be fine-tuned with a more diverse dataset to improve accuracy. The polling mechanism should be replaced with WebSocket or MQTT for real-time alert delivery, and a cooldown mechanism should be added to reduce repeated alerts. External notifications such as Telegram or email can also be added to ensure caregivers are informed even when not actively monitoring the dashboard.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project presented the development of a vision-based AI-IoT robotic car for human-following, fall detection, and near real-time alert monitoring. The system was designed as an end-to-end IoT prototype that integrates hardware sensing, edge AI processing, embedded vehicle control, backend services, database storage, and a web-based dashboard.

The proposed system uses a Pi Camera V3 to capture visual data and a Raspberry Pi 5 as the edge processing device. YOLO-based models are applied to detect a person and classify human states such as standing, sitting, and fallen. Based on the AI results, the Raspberry Pi generates vehicle commands and sends them to an Arduino Uno R3 through Serial UART. The Arduino then controls the DC motor and steering servo through the L298N motor driver.

In addition to local vehicle control, the system also supports IoT event reporting and alert management. When important events such as FALL_DETECTED or PERSON_LOST occur, the Raspberry Pi sends structured event data to the NestJS backend through HTTP REST API. The backend stores events and alerts in a SQLite database using Prisma ORM, while the Next.js dashboard allows users to monitor device status, view recent events, and manage alerts.

Overall, the project demonstrates how IoT architecture and protocols can be applied to build a practical AI-based robotic system. It combines sensing, edge computing, embedded control, communication protocols, backend processing, security mechanisms, and dashboard monitoring into one complete prototype.

5.2 Practical Applicability of the System

The proposed system has practical applicability in several human-support and safety-monitoring scenarios. In home-care environments, the robot can be used as an assistive monitoring device for elderly people, patients, or people living alone. By

following the target person and detecting abnormal states such as falling, the system can help reduce the risk of delayed response when no caregiver is nearby.

In healthcare or rehabilitation facilities, the system can support staff by monitoring patients who may have limited mobility or a high risk of falling. Instead of requiring continuous manual observation, the robot can automatically detect important events and report them to a monitoring dashboard.

The system can also be extended to workplace safety applications. In industrial or warehouse environments, a similar robot could monitor workers in restricted or hazardous areas. When a worker falls or becomes lost from the monitoring view, the system can generate an alert for supervisors.

Besides these applications, the project is also useful as an educational prototype. It demonstrates how different IoT layers work together in a real system, including perception, edge AI processing, embedded control, communication, backend service, database storage, dashboard visualization, and security.

5.3 Future Development Directions

In future development, the system can be improved in several directions. First, the AI model can be trained with a larger and more diverse dataset to improve the accuracy of human state classification under different lighting conditions, camera angles, and real-world postures. More fall scenarios should be included to reduce false negatives and improve system reliability.

Second, the edge AI pipeline can be optimized to improve FPS and reduce latency. Possible improvements include model quantization, input resolution optimization, frame-skipping strategies, or using an accelerator such as Google Coral TPU or Raspberry Pi AI HAT for faster inference.

Third, the communication system can be upgraded from dashboard polling to WebSocket or MQTT. This would allow alerts and device status to be pushed to the dashboard in real time, reducing unnecessary repeated API requests and improving monitoring responsiveness.

Fourth, the robotic platform can be enhanced with additional sensors such as ultrasonic sensors, LiDAR, or depth cameras. These sensors can improve obstacle avoidance, distance estimation, and navigation safety, especially in indoor environments with furniture or moving people.

Fifth, the dashboard can be extended with more practical features, such as mobile push notifications, alert history filtering, device location tracking, user activity logs, and caregiver notification functions. These features would make the system more suitable for real deployment.

Overall, future improvements should focus on increasing AI accuracy, reducing response latency, improving real-time communication, enhancing robot navigation, and expanding the monitoring dashboard. These directions can help transform the current prototype into a more reliable and practical assistive IoT system.

REFERENCES

World Health Organization. (2021). *Falls*. https://www.who.int/news-room/fact-sheets/detail/falls

International Telecommunication Union. (2012). *ITU-T Recommendation Y.2060: Overview of the Internet of Things*. https://www.itu.int/ITU-T/recommendations/rec.aspx?rec=y.2060

Raspberry Pi Ltd. (2023). *Raspberry Pi 5 product brief* (RP-008348-DS). https://pip.raspberrypi.com/documents/RP-008348-DS-raspberry-pi-5-product-brief.pdf

Raspberry Pi Ltd. (2023). *Camera Module 3 product brief*. https://datasheets.raspberrypi.com/camera/camera-module-3-product-brief.pdf

Raspberry Pi Ltd. (2024). *The Raspberry Pi camera guide*. https://www.raspberrypi.com/documentation/accessories/camera.html

Arduino. (2024). *Arduino Uno Rev3 documentation*. https://docs.arduino.cc/hardware/uno-rev3

STMicroelectronics. (2000). *L298: Dual full-bridge driver datasheet*. https://www.st.com/resource/en/datasheet/l298.pdf

Ultralytics. (2024). *Ultralytics YOLO documentation*. https://docs.ultralytics.com/

Prisma. (2024). *Prisma documentation*. https://www.prisma.io/docs

NestJS. (2024). *NestJS documentation*. https://docs.nestjs.com/

NestJS. (2024). *NestJS recipe: Prisma*. https://docs.nestjs.com/recipes/prisma

SQLite Consortium. (2024). *SQLite documentation*. https://sqlite.org/docs.html

Vercel. (2024). *Next.js documentation*. https://nextjs.org/docs

Fielding, R., Nottingham, M., & Reschke, J. (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc9110

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc7519

OWASP Foundation. (2024). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OASIS. (2019). *MQTT version 5.0*. https://www.oasis-open.org/standard/mqtt-v5-0-os/